Índice

0. Cómo usar este apunte

1. El mapa general y el kit de Linux

2. Red: IP, ruteo, ARP y descubrimiento

3. HTTP: el protocolo de la web

4. nginx: servidor web y proxy reverso

5. Criptografía, TLS y certificados

6. DNS: cómo funciona y cómo consultarlo

7. BIND9: ser autoridad de una zona

8. Correo: SMTP, POP3, MIME y antispam

9. DHCP: configuración automática de red

10. SSH: login remoto y túneles

11. Wireshark y análisis de capturas

12. Respuestas de la práctica

Chuleta final: comandos para tener a mano

> **🗣 La fórmula para explicar cualquier ejercicio**
>
> **1.** Qué me piden, en mis palabras. **2.** Qué protocolo o mecanismo entra en juego y cómo funciona en una oración. **3.** Qué hice y por qué cada paso. **4.** Cómo verifico que funciona (y lo muestro). Si sabés decir estas cuatro cosas, tenés el ejercicio.

Qué toman: el mapa de los parciales

Revisando los parciales 2020–2025 que tenés, los ejercicios caen siempre en estas familias. Al lado está el capítulo donde se resuelve cada una.

Tipo de ejercicio	Ejemplos reales	Cap.
Descubrir cosas en la red	MAC de `esmerald` desde `pampero`; puertos abiertos del gateway; hosts y servicios detrás de un gateway, routers intermedios	2
Cliente HTTP con curl	Obtener `text/plain` en español de `/foo/` (con redirect, Basic auth y proxy SOCKS); obtener un 304	3
Servidor nginx	Virtual hosts `itba.example.org` / `www.example.org` /«What?»; proxy reverso `foo.example.org → foo.leak.com.ar`; compresión gzip; conservar un header	4
HTTPS / certificados	`https://seguro.protos.foo` sirviendo `parcial.protos.foo`; guía: mkcert para `foo`	5
DNS cliente	IPs de los MX de `proto.leak.com.ar`; política SPF; cadena de CNAME; autoridad de una zona	6
BIND9	Zona `demiloo.com.ar`; zona `carlitos.com.ar`; arreglar un archivo de zona roto (2Q2025)	7
Correo	Mandar un mail «a mano» con nc cumpliendo MIME, base64 y headers; leerlo por POP3; sacar una imagen o un texto de un pcap	8
DHCP	Configurar isc-dhcp-server con reserva por MAC (2Q2025); IP asignada en un pcap	9
SSH	Entrar con una llave con passphrase; seguir `tail -f` de un archivo remoto; dar Internet a un host sin NAT con túneles	10
Análisis de pcaps	`botica.pcap`, `dhcp-parcial.pcap`, `parcial-correo.pcap` (resueltos con los valores reales)	11

1 El mapa general y el kit de Linux

Las capas, en una página

Cada protocolo resuelve un problema y se apoya en el de abajo. Cuando algo no anda, se diagnostica de abajo hacia arriba: ¿tengo IP? ¿llego al gateway? ¿resuelvo el nombre? ¿el puerto está abierto? ¿el servidor responde lo que espero?

Capa	Qué resuelve	Protocolos de la materia	Dirección que usa
Aplicación	Lo que el usuario quiere hacer: web, mail, nombres, configuración, login remoto	HTTP, SMTP, POP3, IMAP, DNS, DHCP, SSH, Telnet, RIP	URL, nombre de dominio, dirección de mail
Transporte	Llevar bytes entre *procesos*: confiable y ordenado (TCP) o «tiro y olvido» (UDP)	TCP, UDP (SCTP)	Puerto (0–65535)
Red	Llevar paquetes entre *hosts* de distintas redes, saltando por routers	IP, ICMP (ping, traceroute)	Dirección IP
Enlace	Llevar tramas entre equipos de la *misma* red local	Ethernet, Wi-Fi, ARP	Dirección MAC

> 💡 **Las tres direcciones**
>
> Para mandarle algo a `www.ejemplo.com` tu máquina necesita: el **nombre → IP** (lo resuelve DNS), la **IP del siguiente salto** (lo dice la tabla de ruteo: si el destino no está en mi red, el siguiente salto es el gateway) y la **MAC de ese siguiente salto** (lo resuelve ARP). La MAC solo tiene sentido dentro de la red local; la IP de destino viaja igual de punta a punta.

Puertos que tenés que saber de memoria

Puerto	Protocolo	Puerto	Protocolo
22/tcp	SSH	110/tcp · 995	POP3 · POP3 sobre TLS
23/tcp	Telnet (todo en texto plano)	143/tcp · 993	IMAP · IMAP sobre TLS
25/tcp	SMTP entre servidores (MTA→MTA)	443/tcp	HTTPS (HTTP sobre TLS)
587/tcp · 465	SMTP submission (cliente→servidor, con auth) · SMTPS	520/udp	RIP
53/udp y tcp	DNS (TCP para respuestas grandes y transferencias de zona)	1080/tcp	SOCKS (proxy) por convención
67/udp · 68/udp	DHCP servidor · DHCP cliente	8080, 9090…	Puertos «inventados» para pruebas
80/tcp	HTTP	13, 37, 7	daytime, time, echo (servicios viejos de la guía)

Los puertos menores a 1024 son «privilegiados»: solo root puede abrir un socket que escuche ahí. Por eso `nc -l 80` falla sin sudo y `nc -l 9090` no.

Kit de supervivencia en Linux

Estos comandos aparecen en casi todos los ejercicios. Conviene saber qué hace cada uno sin pensarlo.

Servicios (servidores que corren en segundo plano)

```
sudo systemctl status nginx      # ¿está corriendo? ¿falló? muestra las últimas líneas del log
sudo systemctl start nginx       # lo arranca
sudo systemctl stop nginx        # lo detiene (útil para mostrar "el servidor no está
corriendo" en el video)
sudo systemctl restart nginx     # lo detiene y lo vuelve a arrancar: relee toda la
configuración
sudo systemctl reload nginx      # relee la config SIN cortar conexiones (si el servicio lo
soporta)
sudo systemctl enable nginx      # que arranque solo al prender la máquina
journalctl -u nginx -e           # log del servicio (-e salta al final); -f para seguirlo en
vivo
sudo tail -f /var/log/syslog     # el log general del sistema, en vivo: ideal en una terminal
aparte
```

> 💡 **El ciclo de cualquier servidor**
>
> Editar la config → **validarla** (`nginx -t`, `named-checkconf`, `named-checkzone`, `dhcpd -t`) → reiniciar → mirar el log si algo falla → probar con un cliente. El paso de validar te ahorra la mitad de los problemas: te dice el archivo y la línea del error.

Archivos y permisos

```
sudo nano /etc/hosts             # editor simple: Ctrl+O guarda, Ctrl+X sale
ln -s ../sites-available/foo foo # crea un "acceso directo" (link simbólico) llamado foo que
apunta al otro archivo
ls -l                            # lista con permisos: -rw------- = solo el dueño lee/escribe
(600)
chmod 600 llave.pem              # permisos: dueño rw (6), grupo nada (0), otros nada (0)
chmod og-rwx llave.pem           # lo mismo dicho de otra forma: a "otros" y "grupo" sacales r,
w y x
grep -ri boot /var/log           # busca "boot" (-i sin importar mayúsculas) recursivamente (-
r)
tail -f archivo                  # muestra el final del archivo y se queda esperando líneas
nuevas
cat archivo | base64 -d > out    # decodifica base64; | pasa la salida de un comando a otro, >
la manda a un archivo
sha256sum video.mp4              # huella del archivo (se usaba para entregar el parcial)
```

Los permisos se leen en tres grupos de `rwx`: dueño, grupo, otros. En números r=4, w=2, x=1, entonces `600` = rw- --- ---, `644` = rw- r-- r--, `755` = rwx r-x r-x.

Red: ¿quién soy, por dónde salgo, quién está al lado?

Pregunta	Comando nuevo (iproute2)	Comando viejo (net-tools)
¿Qué IP y MAC tienen mis interfaces?	`ip addr` (o `ip a`)	`ifconfig`
¿Cuál es mi tabla de ruteo / mi gateway?	`ip route`	`route -n`
¿Qué MAC tiene cada IP vecina? (tabla ARP)	`ip neigh`	`arp -n`
¿Qué puertos están escuchando en mi máquina y qué proceso?	`sudo ss -ntlp`	`sudo netstat -ntlp`
¿Qué servidor DNS uso?	`resolvectl status`	`cat /etc/resolv.conf`

Los flags de `ss` / `netstat`: `-n` números (no traduce puertos a nombres), `-t` TCP, `-u` UDP, `-l` solo los que escuchan (listen), `-p` muestra el proceso. El de `route` / `arp`: `-n` no intenta resolver nombres (más rápido y muestra IPs).

El archivo /etc/hosts

Antes de preguntarle al DNS, el sistema mira `/etc/hosts`. Es una tabla «IP → nombres» local. Se usa muchísimo en los parciales para que un nombre inventado (`foo.example.org`, `seguro.protos.foo`) apunte a tu propia máquina, donde corre tu nginx.

```
127.0.0.1   localhost foo bar foo.example.org itba.example.org www.example.org
# formato: IP  nombre1 nombre2 ...   (un nombre puede aparecer en una sola línea)
```

> **⚠ Trampa**
>
> `/etc/hosts` afecta solo a *tu* máquina (curl, el navegador, ping). **No** lo ven `dig` ni `nslookup`, porque esas herramientas le preguntan directo al servidor DNS. Si `curl foo.example.org` anda y `dig foo.example.org` no, es normal.

netcat (nc): la navaja suiza

netcat abre una conexión TCP (o UDP) y conecta tu teclado con el socket: lo que escribís se manda, lo que llega se imprime. Sirve para «hablar» a mano cualquier protocolo de texto (HTTP, SMTP, POP3) y para armar servidores de juguete.

```
nc ejemplo.com 80                # cliente TCP: me conecto al puerto 80
nc -C mail.ejemplo.com 25        # -C: cada Enter manda CR LF (\r\n), que es lo que exigen HTTP
y SMTP
nc -l 9090                       # servidor: escucho en el 9090 (en algunas versiones: nc -l -p
9090)
nc -u 10.0.0.1 7                 # UDP en vez de TCP
ncat -lk 8080 -c 'cat'           # ncat (de nmap): -k acepta muchas conexiones, -c pasa cada
conexión por un comando (acá un servidor echo)
nc -v -X 5 -x localhost:1080 host 80   # conectarse pasando por un proxy SOCKS5
```

💡 ¿Por qué CR LF?

HTTP, SMTP y POP3 definen que cada línea termina con los bytes `\r\n` (CR=13, LF=10). Linux usa solo `\n`. Algunos servidores toleran solo LF y otros no, por eso conviene `nc -C` o armar el pedido con `printf '...\r\n'`.

✍ Practicá en papel — Capítulo 1

1. ¿Qué permisos numéricos tiene un archivo `-rw-r-----`? ¿Y uno que querés que solo vos puedas leer y escribir?
2. Agregaste `127.0.0.1 foo` a `/etc/hosts`. ¿Qué devuelve `ping foo`? ¿Y `dig foo`? ¿Por qué son distintos?
3. ¿Qué comando usás para saber si nginx está escuchando en el puerto 80 de tu máquina?
4. ¿Por qué `nc -l 80` falla y `sudo nc -l 80` no?
5. Ordená el diagnóstico de «no me anda `curl http://foo.example.org`» de abajo hacia arriba.

2 Red: IP, ruteo, ARP y descubrimiento

IP, máscara y gateway

Una dirección IPv4 son 32 bits escritos como cuatro números (`192.168.100.25`). La **máscara** dice qué parte es «la red» y qué parte es «el host». `/24` = `255.255.255.0` = los primeros 24 bits son la red. Entonces `192.168.100.25/24` vive en la red `192.168.100.0`, cuyos hosts van del `.1` al `.254` (el `.0` es la red y el `.255` el broadcast).

Prefijo	Máscara	Direcciones	Hosts usables
/24	255.255.255.0	256	254
/25	255.255.255.128	128	126
/27	255.255.255.224	32	30
/16	255.255.0.0	65 536	65 534

Redes privadas (no se rutean en Internet, necesitan NAT para salir): `10.0.0.0/8`, `172.16.0.0/12`, `192.168.0.0/16`. `127.0.0.0/8` es loopback: «yo mismo».

Cómo decide un host a dónde mandar un paquete

Mira su **tabla de ruteo** y elige la regla más específica (el prefijo más largo) que contenga al destino.

```
$ route -n
Destination     Gateway         Genmask         Flags Iface
0.0.0.0         10.16.1.254     0.0.0.0         UG    ens3    # ruta default: "todo lo demás"
va al gateway 10.16.1.254
10.16.1.0       0.0.0.0         255.255.255.0   U     ens3    # mi red local: entrego directo
(Gateway 0.0.0.0)
```

Flags: `U` = la ruta está activa (up), `G` = pasa por un gateway. Con `ip route` se ve así: `default via 10.16.1.254 dev ens3`.

```
sudo ip route add 10.0.0.0/24 via 192.168.50.250   # "para llegar a 10.0.0.0/24, mandá al
router 192.168.50.250"
sudo ip route del 10.0.0.0/24                      # borrarla
sudo ip route add 192.168.14.0/27 via 127.0.0.1 reject   # (o: ip route add prohibit
192.168.14.0/27) bloquear una subred
cat /proc/sys/net/ipv4/ip_forward                  # 1 = este host reenvía paquetes ajenos
(actúa de router), 0 = no
echo 1 | sudo tee /proc/sys/net/ipv4/ip_forward    # activarlo
```

> **⚠ Trampa**
>
> El gateway que ponés en una ruta tiene que estar en una red a la que ya estás conectado directamente. No podés decir «para ir a X mandá a un router que está tres redes más allá».

ARP: de IP a MAC

Dentro de una red local las tramas Ethernet se mandan a una **MAC**. Cuando un host necesita la MAC de una IP de su red, manda un **ARP Request** en broadcast («¿Quién tiene 10.16.1.102? Avisale a 10.16.1.5») y el dueño contesta con un **ARP Reply** unicast con su MAC. La respuesta se guarda un rato en la **tabla ARP** (caché).

```
$ arp -n
Address         HWtype  HWaddress           Flags Iface
10.16.1.254     ether   00:87:31:e1:0f:d2   C     ens3     # C = aprendida dinámicamente (cache)
10.16.1.102     ether   00:16:3e:bb:e3:3c   C     ens3
```

> 💡 **Por qué hacer ping antes de mirar la tabla ARP**
>
> Si la IP no aparece en la tabla es porque tu host no le habló hace poco. Un `ping` obliga al host a mandar un paquete IP a esa IP, para eso necesita la MAC, entonces dispara el ARP Request. Después del ping la entrada aparece. Funciona aunque el host bloquee el ping: el ARP se resuelve igual.

> ⚠ **Trampa conceptual**
>
> Solo podés ver la MAC de hosts de **tu misma red**. Si el destino está detrás de un router, en tu tabla ARP vas a ver la MAC del router, no la del destino. Por eso los enunciados dicen «desde la interfaz `ens3` de pampero»: hay que estar en la misma LAN que el host buscado.

ping y traceroute

ping manda ICMP *Echo Request* y espera *Echo Reply*. Prueba que hay ida y vuelta a nivel IP. Que no responda no significa que el host esté apagado: puede tener ICMP filtrado.

traceroute descubre los routers del camino aprovechando el campo **TTL** (Time To Live) del header IP. Cada router le resta 1 al TTL, y si llega a 0 descarta el paquete y le avisa al origen con un ICMP **Time Exceeded** (tipo 11), firmado con su propia IP.

1. Manda paquetes con TTL=1: el primer router los descarta y responde Time Exceeded → ya sé quién es el salto 1.
2. TTL=2 → responde el salto 2. Y así sucesivamente (3 sondas por salto, por eso ves tres tiempos).
3. En Linux las sondas son UDP a puertos altos (33434, 33435…). Cuando una llega al destino final, ese puerto no tiene a nadie escuchando, entonces el destino responde ICMP **Port Unreachable** (tipo 3): fin del recorrido.

```
traceroute -n 10.0.0.1     # -n: no resolver nombres (más rápido, se ven las IPs)
traceroute -I 8.8.8.8      # -I: sondas ICMP Echo en vez de UDP (pasa firewalls que bloquean
UDP)
traceroute -T -p 80 host   # -T: sondas TCP SYN al puerto 80 (lo que más pasa por firewalls)
traceroute host 4000       # paquetes de 4000 bytes: se fragmentan (MTU de Ethernet = 1500)
```

Las líneas con `* * *` son routers que no contestan Time Exceeded (filtran ICMP): el salto existe, pero no se identifica.

nmap: qué hosts hay y qué puertos tienen abiertos

Para saber si un puerto TCP está abierto, nmap intenta empezar el **three-way handshake** (SYN → SYN/ACK → ACK):

- Si responde **SYN/ACK** → hay un proceso escuchando: **open**.
- Si responde **RST** → el host existe pero nadie escucha ahí: **closed**.
- Si no responde nada o llega un ICMP de error → un firewall lo tira: **filtered**.

Flag	Qué hace	Cuándo usarlo
`-sT`	Connect scan: completa el handshake usando la llamada `connect()` del sistema	Sin root (en pampero no sos root). Queda registrado en los logs del destino
`-sS`	SYN scan («half-open»): manda SYN, si vuelve SYN/ACK responde RST sin completar	Con root. Más rápido y más «silencioso». Es el default con sudo
`-sU`	Escaneo UDP: si vuelve ICMP Port Unreachable está cerrado, si no vuelve nada es «open|filtered»	Lento e impreciso, porque UDP no tiene handshake
`-sV`	Se conecta y lee el «banner» para adivinar servicio y versión	Para decir «tiene expuesto nginx 1.18 en el 80»
`-p-`	Escanea los 65 535 puertos (por defecto solo los 1000 más comunes)	Cuando piden «todos los puertos abiertos»
`-p 22,80`	Solo esos puertos	Rapidez
`-Pn`	No hace el ping previo: asume que el host está vivo	Cuando el host no responde ping pero sí tiene puertos
`-sn`	Solo descubre hosts vivos, no escanea puertos («ping sweep»)	`nmap -sn 10.0.0.0/24` para ver quién hay en una red
`-O`	Adivina el sistema operativo por cómo responde la pila TCP/IP	Ejercicio E78 de la guía

Ejercicios de parciales resueltos

Parcial 1C2020 / 2025 — Ejercicio I

Encuentre la dirección MAC del host `esmerald.it.itba.edu.ar` desde la interfaz `ens3` del host `pampero.it.itba.edu.ar`.

```
ssh usuario@pampero.itba.edu.ar      # me paro en pampero, que está en la misma LAN que esmerald
dig +short esmerald.it.itba.edu.ar   # nombre -> IP. Respuesta de ejemplo: 10.16.1.102
ping -c 1 10.16.1.102                # fuerzo el ARP por si la MAC no estaba en la caché (-c 1: un solo paquete)
arp -n | grep 10.16.1.102            # leo la MAC: 00:16:3e:bb:e3:3c   (o: ip neigh show 10.16.1.102)
```

🗣 Qué decir

«La MAC es una dirección de capa de enlace, entonces solo la puedo conocer desde un host de la misma red. Me conecto a pampero, resuelvo la IP de esmerald con DNS, hago un ping para que pampero dispare un ARP Request por la interfaz ens3, y leo la respuesta en la tabla ARP.»

Parcial 1C2020 — Ejercicio II

Enumere todos los puertos abiertos TCP del gateway del host `pampero.it.itba.edu.ar`.

```
ssh usuario@pampero.itba.edu.ar
route -n                        # la línea 0.0.0.0 (default) tiene el gateway: 10.16.1.254
nmap -sT -p- 10.16.1.254        # -sT porque no soy root en pampero; -p- porque pide TODOS los
puertos
# Resultado de ejemplo: 22/tcp ssh, 23/tcp telnet, 80/tcp http, 443/tcp https
```

Parcial 2Q2025 — Ejercicio II (y 2024-2C Ejercicio III)

El host en `192.168.50.250` es el gateway de la red `10.0.0.0/24`. Sabe que existe un servidor distinto al gateway en esta red pero no sabe qué IP tiene. Descubra la dirección del servidor y los servicios que tiene expuestos. Debe mostrar el resultado de acceder vía HTTP a dicho servidor y los routers que intervienen. (VM en modo Bridge a la red del laboratorio.)

```
ip addr                                       # 1. confirmo que mi VM tiene IP en la red del
laboratorio (bridge)
ping -c 2 192.168.50.250                      # 2. llego al gateway (está en mi red)
ip route                                      # 3. veo que NO tengo ruta a 10.0.0.0/24: iría
por mi default y se perdería
sudo ip route add 10.0.0.0/24 via 192.168.50.250   # 4. le enseño a mi host el camino
nmap -sn 10.0.0.0/24                          # 5. barrido: qué IPs responden en esa red
(descarto la del gateway)
sudo nmap -sS -sV -Pn -p- 10.0.0.X            # 6. puertos abiertos y versiones del servidor
encontrado
curl -v http://10.0.0.X/                      # 7. el acceso HTTP que piden mostrar (-v para
ver request y response)
traceroute -n 10.0.0.X                        # 8. los routers: debería aparecer
192.168.50.250 y después el destino
```

🗣 Qué decir

«Mi host no sabía cómo llegar a 10.0.0.0/24, así que agregué una ruta estática indicando que el siguiente salto es 192.168.50.250, que está en mi red. Con un barrido de nmap encontré qué host además del gateway responde, después escaneé sus puertos con detección de versión y accedí con curl al servicio HTTP. Traceroute muestra los saltos porque va subiendo el TTL y cada router responde con ICMP Time Exceeded.»

⚠ Si no responde

Si el gateway no tiene ruta de vuelta hacia tu red, el servidor no te puede contestar. Si `nmap -sn` no encuentra nada, probá `-Pn` con algunos puertos comunes (`-p 22,80,443`): puede ser que el ping esté filtrado.

✍ Practicá en papel — Capítulo 2

1. Tu host es `192.168.1.20/24` con gateway `192.168.1.1`. ¿A qué MAC manda la trama de un paquete dirigido a `192.168.1.33`? ¿Y uno dirigido a `8.8.8.8`?
2. En un nmap ves `22/tcp filtered`. ¿Qué pasó a nivel de paquetes?
3. Un traceroute muestra 3 líneas con IPs y en la cuarta el destino. ¿Cuántos ICMP Time Exceeded recibió tu host (con 3 sondas por salto)? ¿Qué ICMP recibió del destino?
4. ¿Por qué en pampero usás `-sT` y en tu VM podés usar `-sS`?
5. Escribí el comando para que tu host llegue a `172.20.0.0/16` a través del router `192.168.1.254`.

3 HTTP: el protocolo de la web

La idea

HTTP es un protocolo **cliente-servidor, de texto y sin estado**. El cliente (el *user agent*: navegador, curl, wget) abre una conexión TCP al puerto 80 del servidor, manda un **request** y recibe un **response**. «Sin estado» significa que cada pedido se entiende solo: el servidor no recuerda el anterior. Cuando hace falta «recordar» (sesiones, logins) se usan **cookies** o headers como `Authorization` que el cliente vuelve a mandar en cada pedido.

Lo que pedís es un **recurso** (identificado por una URL) y lo que recibís es una **representación** de ese recurso. El mismo recurso `/lang` puede tener varias representaciones: en español o en inglés, en HTML o en texto plano, comprimida o no. El cliente dice qué prefiere con headers `Accept-*` y el servidor elige: eso es la **negociación de contenido**, y es lo que toman en el ejercicio «obtenga la representación text/plain en español».

Anatomía de una URL

```
https://usuario:clave@www.ejemplo.com:8443/docs/rfc.html?q=dns&p=2#seccion-3
└──┘   └──────────┘ └──────────────┘ └──┘└──────────────┘└─────────┘└─────────┘
scheme   userinfo          host      port      path         query    fragment
```

- **scheme**: el protocolo. Define el puerto por defecto (http→80, https→443).
- **host**: se resuelve por DNS, y además viaja en el header `Host` (clave para los virtual hosts).
- **path + query**: es lo que va en la primera línea del request.
- **fragment** (`#seccion-3`): **no se envía al servidor**. Lo usa solo el navegador para saltar a una parte de la página.
- Los caracteres raros se escriben con **percent-encoding**: `%7E` = `~`, `%20` = espacio. El host no distingue mayúsculas, el path sí (en general). `:80` en http es lo mismo que no poner puerto.

El formato de los mensajes

```
GET /foo/ HTTP/1.1                  # método, path, versión
Host: ejercicio2.proto.leak.com.ar  # obligatorio en HTTP/1.1
User-Agent: curl/8.5.0
Accept: text/plain                  # quiero texto plano
Accept-Language: es                 # en español
Accept-Encoding: gzip               # puedo recibir gzip
                                    # línea vacía = fin de headers
```

```
HTTP/1.1 200 OK                     # versión, código, frase
Date: Wed, 19 May 2021 02:18:51 GMT
Server: nginx/1.18.0
Content-Type: text/plain; charset=utf-8
Content-Language: es
Content-Length: 27                  # cuántos bytes de cuerpo
                                    # línea vacía
Hola, esta es la versión en español
```

Cada línea termina en `\r\n` y una línea vacía separa headers de cuerpo. Los nombres de headers **no distinguen mayúsculas** (`Accept` = `AccEpT`), sus valores a veces sí.

¿Cómo sabe el cliente dónde termina el cuerpo?

- `Content-Length: N`: lee exactamente N bytes.
- `Transfer-Encoding: chunked`: el cuerpo viene en pedazos, cada uno precedido por su tamaño en hexadecimal, y termina con un pedazo de tamaño `0`. Sirve cuando el servidor no sabe el tamaño de antemano (contenido generado al vuelo o comprimido en streaming).
- Cerrando la conexión (HTTP/1.0 o `Connection: close`).

Conexiones persistentes

En HTTP/1.1 la conexión TCP queda abierta por defecto para varios pedidos (*keep-alive*), evitando un handshake TCP (y TLS) por cada recurso. Requisito: que cada mensaje tenga su largo bien delimitado (Content-Length o chunked). **Pipelining** es mandar varios pedidos sin esperar las respuestas; casi nadie lo usa. HTTP/2 lo resuelve mejor multiplexando varios pedidos en una sola conexión.

Métodos

Método	Para qué	Seguro	Idempotente
GET	Obtener una representación	Sí	Sí
HEAD	Igual que GET pero sin cuerpo (solo headers)	Sí	Sí
POST	Procesar datos: crear, enviar formulario, acción con efectos	No	No
PUT	Reemplazar el recurso con lo que mando	No	Sí
DELETE	Borrar el recurso	No	Sí
PATCH	Modificar parcialmente	No	No (en general)
OPTIONS	Qué métodos admite el recurso	Sí	Sí
TRACE	El servidor devuelve el request tal como le llegó (detectar proxies que lo modifican)	Sí	Sí
CONNECT	Pedirle a un proxy que abra un túnel TCP (se usa para HTTPS a través de proxies)	No	No

> 💡 **Seguro vs idempotente**
>
> **Seguro** = no cambia nada en el servidor (solo lectura). **Idempotente** = hacerlo 1 o 10 veces deja el servidor en el mismo estado. DELETE es idempotente aunque borre: después de la primera vez el recurso no existe, y las siguientes lo dejan igual de borrado (aunque la respuesta sea 404). Por eso un «acelerador» que precarga todos los links con GET destruye un ABM que borra con `GET /items/123/delete`: el culpable es el desarrollador, que usó un método seguro para algo con efectos. Debería ser POST o DELETE.

Códigos de estado

Código	Significado y cuándo aparece
200 OK	Todo bien, viene la representación
201 Created · 204 No Content	Se creó el recurso (con `Location`) · ok sin cuerpo
206 Partial Content	Respuesta a un pedido con `Range` (reanudar descargas: `wget -c`)
301 Moved Permanently · 308	Se mudó para siempre. El header `Location` dice a dónde. 308 = igual pero obliga a repetir el mismo método
302 Found · 303 · 307	Redirección temporal (307 conserva el método; 303 = «andá a buscar el resultado con GET»)
304 Not Modified	Respuesta a un GET condicional: «tu copia en caché sigue sirviendo», sin cuerpo
400 Bad Request	Pedido mal formado (falta Host en 1.1, parámetro con formato inválido)
401 Unauthorized	No estás autenticado. Viene con `WWW-Authenticate` diciendo cómo autenticarte
403 Forbidden	Sé quién sos (o no importa) y no tenés permiso
404 · 410 Gone	No existe · existió y se borró a propósito, no va a volver (los buscadores lo desindexan)
405 Method Not Allowed	El recurso no admite ese método (viene con `Allow: GET`)
406 Not Acceptable	No tengo ninguna representación que cumpla tus `Accept*`
409 Conflict · 412 Precondition Failed	Conflicto de edición · falló un `If-Match` / `If-Unmodified-Since` (alguien editó antes que vos)
415 Unsupported Media Type	Me mandaste un cuerpo en un formato que no acepto
500 · 502 · 503 · 504	Error interno · el proxy recibió una respuesta inválida del de atrás · servicio no disponible · el de atrás no contestó a tiempo

Headers que tenés que conocer

Negociación de contenido

```
Accept: text/html, text/plain;q=0.8, */*;q=0.1   # tipos que acepto; q = preferencia de 0 a 1 (default 1)
Accept-Language: es-AR, es;q=0.9, en;q=0.5       # idiomas en orden de preferencia
Accept-Encoding: gzip, deflate, br               # compresiones que sé descomprimir
Accept-Charset: utf-8                            # (casi en desuso)
```

El servidor responde con `Content-Type`, `Content-Language`, `Content-Encoding: gzip` y agrega `Vary: Accept-Language` para avisar a los cachés que la respuesta depende de ese header.

Un **media type** tiene la forma `tipo/subtipo; parámetros`: `text/html`, `text/plain; charset=utf-8`, `text/css`, `application/javascript`, `image/jpeg`, `image/png`, `image/gif`, `application/pdf`, `application/json`, y para un binario cualquiera `application/octet-stream`.

Autenticación Basic

```
Authorization: Basic dXNlcm5hbWU6cGFzc3dvcmQ=     # base64("username:password")
echo dXNlcm5hbWU6cGFzc3dvcmQ= | base64 -d        # -> username:password   (¡no es cifrado, es solo codificación!)
curl -u username:password http://...              # curl arma el header solo
```

> ⚠ **Para el oral**
>
> Base64 **no protege nada**: cualquiera que snifee la red lee la password (ejercicio E30: `YWxndW51c3VhcmlvOmFsZ3VuYXBhc3N3b3Jk` = `algunusuario:algunapassword`). Basic solo es aceptable sobre HTTPS.

Caché y pedidos condicionales

Para ahorrar tráfico, el servidor manda junto a la representación datos para validarla después:

- `Last-Modified: Sun, 22 Apr 2018 23:52:26 GMT` → el cliente después pregunta `If-Modified-Since: <esa fecha>`.
- `ETag: "5e8a-1b2c"` (una «huella» de la versión) → el cliente pregunta `If-None-Match: "5e8a-1b2c"`.
- Si no cambió, el servidor responde **304 Not Modified sin cuerpo**. Si cambió, 200 con la nueva versión.
- `Cache-Control: max-age=3600, must-revalidate` = «podés usar tu copia 1 hora sin preguntar; pasado eso, revalidá sí o sí antes de usarla».
- Un ETag **«shallow»** se calcula sobre los bytes ya generados (ahorra ancho de banda pero no CPU del servidor). Uno **«deep»** se calcula sin generar la respuesta (por ejemplo, con la versión en la base de datos) y ahorra las dos cosas.

```
curl -sI http://example.com/                          # -I = HEAD: miro ETag y Last-Modified
curl -i -H 'If-None-Match: "84238dfc8092e5d9c0dac8ef93371a07"' http://example.com/   # -> HTTP/1.1 304 Not Modified
curl -i -H 'If-Modified-Since: Thu, 17 Oct 2019 07:18:26 GMT' http://example.com/
```

(Los valores de ETag y fecha son de ejemplo: siempre copiá los que te devolvió el HEAD.)

Otros headers frecuentes

`Location`	A dónde redirigir (con 3xx) o dónde quedó lo creado (201)
`Set-Cookie` / `Cookie`	El servidor guarda un dato en el cliente; el cliente lo devuelve en cada pedido. Así se «recuerda» la sesión en campus
`Content-Disposition: attachment; filename="x.pdf"`	«Descargalo en vez de mostrarlo»
`Range: bytes=1000-` / `Accept-Ranges` / `Content-Range`	Descarga parcial para reanudar (`wget -c`). Solo funciona si el servidor lo soporta
`Via`, `X-Forwarded-For`	Los agregan los proxies: por cuáles pasó y cuál era la IP original del cliente
`Strict-Transport-Security`	HSTS: «de ahora en más, a este dominio entrá solo por HTTPS»

curl: el cliente HTTP del parcial

Flag	Qué hace
`-v`	Verbose: muestra el request (`>`), los headers de respuesta (`<`), la resolución DNS y el handshake TLS. El que más vas a usar para explicar
`-i`	Incluye los headers de respuesta en la salida
`-I`	Hace HEAD (solo headers)
`-L`	Sigue las redirecciones (3xx con Location)
`-H 'Nombre: valor'`	Agrega o reemplaza un header
`-X METODO`	Cambia el método (`-X DELETE`, `-X TRACE`)
`-d 'a=1&b=2'`	Manda ese cuerpo con POST (Content-Type form-urlencoded)
`-u user:pass`	Autenticación Basic
`-o archivo` / `-O`	Guarda el cuerpo en un archivo / con el nombre del recurso
`-s`	Silencioso (sin barra de progreso)
`--compressed`	Manda `Accept-Encoding` y descomprime la respuesta. Lo que pedían en el ejercicio de gzip
`-0` / `--http1.1` / `--http2`	Fuerza la versión de HTTP
`-x socks5h://host:1080`	Usa un proxy (HTTP o SOCKS). Con `socks5h` el **proxy** resuelve el nombre
`--resolve foo:80:127.0.0.1`	«Hacé de cuenta que foo resuelve a 127.0.0.1» sin tocar /etc/hosts
`-k`	Acepta certificados no confiables. **Prohibido** cuando el enunciado dice «no se permiten otras opciones»

wget es para descargar archivos (no interactivo, reintenta, `-c` reanuda una descarga cortada usando `Range`). **curl** es para hablar HTTP con control total de headers y métodos.

HTTP a mano con netcat

```
printf 'GET / HTTP/1.1\r\nHost: example.com\r\nConnection: close\r\n\r\n' | nc example.com 80
# Sin el Host, un servidor HTTP/1.1 responde 400 Bad Request: el header es obligatorio

nc -l 9090        # servidor de juguete: abrí http://localhost:9090/ en el navegador y vas a
ver el request completo
                  # después escribí a mano la respuesta:
HTTP/1.1 200 OK
Content-Type: text/plain
Content-Length: 5

hola
```

Proxies: forward, reverse y SOCKS

- **Proxy (forward)**: está del lado del cliente. El cliente le pide «traeme http://x.com/». Lo usa el cliente para salir (filtrar, cachear, salir desde otra red).
- **Proxy reverso (gateway)**: está del lado del servidor. El cliente cree que habla con el servidor original, pero el proxy reenvía el pedido a otro servidor de atrás. Sirve para balancear carga, cachear, comprimir, terminar TLS y esconder servidores viejos o inseguros. Es lo que hace nginx con `proxy_pass` (capítulo 4).
- **SOCKS5**: proxy genérico a nivel TCP (no entiende HTTP). El cliente le dice «conectame a host:puerto» y el proxy reenvía bytes. `ssh -D` crea uno (capítulo 10).

> 💡 **socks5 vs socks5h**
>
> Con `socks5://` curl resuelve el nombre **localmente** y le pasa una IP al proxy. Con `socks5h://` le pasa el **nombre** y lo resuelve el proxy. Si el nombre solo existe en la red del proxy (un host interno del ITBA), necesitás la `h`. Esta es la respuesta a «¿por qué `curl pawserver.it.itba.edu.ar -x socks5://pampero.itba.edu.ar:1080` falla desde casa y anda desde el ITBA?»: desde casa tu DNS no puede resolver ese nombre interno (y además el puerto 1080 de pampero puede no ser accesible desde afuera).

Ejercicios de parciales resueltos

Parcial 2021 y Recuperatorios — «text/plain en español»

Utilizando curl obtenga la representación textual (media type `text/plain`) en el lenguaje español (`es`) del recurso `http://ejercicio2.proto.leak.com.ar/foo/`. A su poder llegó un request y una respuesta sobre el mismo servidor con información útil (incluía `Authorization: Basic dXNlcm5hbWU6cGFzc3dvcmQ=`).

La estrategia es ir agregando de a una cosa y leer el código de respuesta cada vez. Cada código te dice qué falta:

```
curl -v http://ejercicio2.proto.leak.com.ar/foo/
#  -> 301/302 con Location: /bar/   => hay que seguir la redirección: agrego -L
curl -v -L http://ejercicio2.proto.leak.com.ar/foo/
#  -> 401 Unauthorized + WWW-Authenticate: Basic   => pide credenciales: uso el header que me
dieron
curl -v -L -H 'Authorization: Basic dXNlcm5hbWU6cGFzc3dvcmQ='
http://ejercicio2.proto.leak.com.ar/foo/
#  -> 200 pero en HTML y/o inglés  => falta negociar tipo e idioma
curl -v -L -H 'Authorization: Basic dXNlcm5hbWU6cGFzc3dvcmQ=' \
     -H 'Accept: text/plain' -H 'Accept-Language: es' \
     http://ejercicio2.proto.leak.com.ar/foo/
#  -> 200, Content-Type: text/plain, Content-Language: es   ✔
```

⚠ Detalles que hacen perder puntos

- El formato del header es `-H 'Accept: text/plain'`. Escribir `-H:'Accept...'` (como aparece en un apunte viejo) está mal.
- Al seguir un redirect a **otro host**, curl deja de mandar el `Authorization` por seguridad. Si el Location apunta a otro dominio, pedí directamente la URL nueva con los headers.
- Si te da **406**, el servidor no tiene esa combinación: revisá que no hayas escrito mal el media type.

🗣 Qué decir

«HTTP permite negociar la representación de un recurso: con `Accept` pido el media type y con `Accept-Language` el idioma. Fui leyendo los códigos: el 301 me indicó seguir la redirección con -L, el 401 me indicó que el recurso exige autenticación Basic, que es usuario:clave en base64, y con los headers de negociación obtuve el texto plano en español, como confirma el Content-Type de la respuesta.»

Parcial 2024-2C — Ejercicio II (con proxy)

Igual al anterior, pero la red no rutea hacia `ejercicio2.sebikul.com`, que sí es accesible mediante un proxy `socks5h://proxy.sebikul.com:1080`.

```
curl -v -x socks5h://proxy.sebikul.com:1080 \
     -H 'Accept: text/plain' -H 'Accept-Language: es' \
     http://ejercicio2.sebikul.com:8080/foo/
# -x: todo el tráfico pasa por el proxy SOCKS; la h hace que el NOMBRE lo resuelva el proxy,
#     porque desde mi red ese host no es alcanzable (y quizás ni resoluble)
```

Ejercicios de repaso

(a) Obtener del host example.com una respuesta HTTP 304. (b) Con ncat, lograr que curl -L tire «too many redirects». (c) Exponer en el puerto X de pampero dos servicios: por 127.0.0.1 debe decir «hola», por 10.16.1.100 debe decir «chau».

```
# (a) 304: primero obtengo el validador y después hago el GET condicional
curl -sI http://example.com/ | grep -i etag
curl -i -H 'If-None-Match: "valor-que-copié"' http://example.com/      # -> 304 Not Modified,
sin cuerpo

# (b) un servidor que siempre redirige a sí mismo
ncat -lk 8080 -c 'printf "HTTP/1.1 302 Found\r\nLocation: http://localhost:8080/\r\nContent-
Length: 0\r\n\r\n"'
curl -L http://localhost:8080/        # -> curl: (47) Maximum (50) redirects followed

# (c) dos sockets en el mismo puerto pero ligados a DISTINTAS direcciones IP
ncat -lk 127.0.0.1   X -c 'echo hola' &    # -l escucha, -k acepta muchas conexiones, -c corre
el comando por conexión
ncat -lk 10.16.1.100 X -c 'echo chau' &    # & lo deja corriendo en segundo plano
ncat 127.0.0.1 X --recv-only          # -> hola
```

💡 Por qué (c) funciona

Un socket que escucha se identifica por **(IP local, puerto)**, no solo por el puerto. Si no indicás IP, escucha en `0.0.0.0` (todas) y ocupa el puerto en todas las interfaces. Al ligar cada uno a una IP distinta, conviven.

✍ Practicá en papel — Capítulo 3

1. ¿Qué código usarías en cada caso? (a) el recurso se mudó para siempre y sabés a dónde; (b) un anónimo encontró la URL del backoffice; (c) el cliente no está autenticado; (d) guardaste una entrada de blog que otro editó mientras tanto; (e) falta un parámetro obligatorio; (f) suben un archivo en un formato que no soportás; (g) el sistema externo del que dependés está caído; (h) mandan DELETE a un recurso que solo admite GET.
2. ¿Qué parte de `https://tools.ietf.org/html/rfc2616#section-14.19` no viaja al servidor? ¿Por qué?
3. Decodificá a mano (o con la tabla del capítulo 8) `Authorization: Basic YTpi`.
4. Escribí el curl para pedir `http://x.com/doc` en inglés, como JSON, siguiendo redirecciones, mostrando los headers.
5. ¿Qué diferencia hay entre recibir 304 y recibir 200 con el mismo contenido? ¿Quién ahorra qué?
6. ¿Por qué `Cache-Control: max-age=3600, must-revalidate` no significa «no uses la caché»?

AMPLIACIÓN DEL CAPÍTULO 3 · HTTP

Headers acumulativos y diagnóstico por código

Muchos ejercicios de `curl` son una conversación de a un paso: pedís, el servidor te rechaza con un código, y vos agregás **una sola cosa** y volvés a pedir. Acá están los headers que se van sumando, qué significa cada uno, y cómo leer el código para saber qué falta.

Los headers que vas agregando, por situación

SI EL SERVIDOR…	AGREGÁS	QUÉ ES / CUÁNDO
exige un navegador ("solo navegadores conocidos")	`-A "Mozilla/5.0"`	`User-Agent`: quién sos como cliente. Por defecto `curl/8.x`; a veces piden uno de navegador (empieza con `Mozilla/5.0`).
pide identificarte con un dato propio	`-H "X-Legajo: 63051"`	Header **personalizado** (los `X-…` los inventa el servidor). `-H "Nombre: valor"` sirve para cualquiera.
quiere la respuesta en español	`-H "Accept-Language: es"`	Negociación de idioma. Da 406 si no lo pedís. Para formato: `-H "Accept: application/json"`.
pide un parámetro en la URL	`"…/ruta?token=valor"`	Query string. El primero con `?`, los siguientes con `&`. **Comillas** en la URL por el `?`.
pide usuario y contraseña (Basic)	`-u admin:admin`	Arma `Authorization: Basic base64(user:pass)`. Lo indica `WWW-Authenticate: Basic`.
dice "tenés que venir de tal página"	`-e http://.../panel`	`Referer`: de qué página venís.
te dio una cookie y la pide de vuelta	`-b "k=v"`	Devuelve la cookie (`Cookie: k=v`). La recibís en `Set-Cookie`.
"solo desde la red interna"	`-H "X-Forwarded-For: 10.0.0.1"`	Header que los proxies usan para informar la IP de origen. Si no alcanza, el filtro mira la IP real: hay que salir por un proxy/túnel.

Leé el código y sabé qué falta

CÓDIGO	QUÉ ESTÁ PASANDO	QUÉ PROBÁS
302 / 301	Redirección: el recurso está en otro lado (header `Location`).	`-L`, o pedir a mano la ruta de `Location`.
400	Falta o está mal un header o un parámetro. Suele decir cuál (en el cuerpo o en un header).	Agregás el header (`-H`) o el parámetro (`?x=…`) que nombra.
401	Falta autenticación. `WWW-Authenticate` dice el tipo.	Basic → `-u u:p` · Bearer → `-H "Authorization: Bearer …"`.
403	Te rechazan por **quién sos**: User-Agent, origen, Referer o un token incorrecto.	`-A`, `-e`, el header/token que corresponda.
404	Ruta inexistente **para ese método**, o host virtual equivocado.	Revisás ruta y método; si hay `server_name`, `-H "Host: nombre"`.

405	La ruta existe pero no con ese método.	Mirás `Allow:` y usás ese método (`-X`).
406	No pueden darte lo que aceptás.	Ajustás `Accept`, `Accept-Language` o `Accept-Encoding`.
407	El **proxy** pide credenciales.	`-U user:pass` (mayúscula = proxy).
`curl: (6)` `(7)(97)`	No hubo respuesta HTTP: no resuelve / no conecta / falló el proxy SOCKS.	Revisás IP, puerto, `-x`. No toques los headers.

💡 **El método, en una frase**

Pedí con `-i` (o `-v` si algo no cierra) → leé el código → agregá **una sola cosa** → repetí. Es **acumulativo**: no saques lo anterior.

⚠️ **Dos trampas**

Si al agregar algo **vuelve un error anterior**, sacaste algo que hacía falta. Si **el error no cambia**, tu header no llegó o el valor no sirve: mirá las líneas `>` con `-v`.

Ejercicio resuelto · búsqueda del tesoro por proxy SOCKS

Un servidor `http://tesoro.lab/` solo es accesible por un proxy SOCKS en `127.0.0.1:1080`, y pide las cosas de a una. Hay que llegar al FLAG con un solo `curl`. Es el patrón del Ej 1 del parcial. Cada paso agrega lo que el código anterior reclamó:

```
# 1) entro por el proxy. socks5h = lo resuelve el proxy (el nombre solo existe "del otro lado")
curl -i -x socks5h://127.0.0.1:1080 http://tesoro.lab/
   → 302 Found · Location: /pista1              ⇒ sigo a esa ruta

# 2) 403: quiere un navegador conocido
curl -i -x socks5h://127.0.0.1:1080 -A "Mozilla/5.0" http://tesoro.lab/pista1
   → 400 Bad Request · "Falta tu identificacion: header X-Legajo"

# 3) agrego el header propio
... -A "Mozilla/5.0" -H "X-Legajo: 63051" ...
   → 406 Not Acceptable · "Prefiero hablar en espanol (Accept-Language)"

# 4) pido español
... -H "X-Legajo: 63051" -H "Accept-Language: es" ...
   → 400 · falta ?token · y en un header viene una pista:  X-Pista: dG9rZW49emFyaWd1ZXlh
echo 'dG9rZW49emFyaWd1ZXlh' | base64 -d        # → token=zarigueya

# 5) agrego el parámetro (comillas por el ?)
... -H "Accept-Language: es" "http://tesoro.lab/pista1?token=zarigueya"
   → 401 Unauthorized · WWW-Authenticate: Basic realm="tesoro"

# 6) comando final: sumo -u y llego al FLAG
curl -i -x socks5h://127.0.0.1:1080 -A "Mozilla/5.0" \
  -H "X-Legajo: 63051" -H "Accept-Language: es" \
  -u admin:admin "http://tesoro.lab/pista1?token=zarigueya"
   → 200 OK · FLAG{tesoro-encontrado-por-el-tunel-socks}
```

🗣 Qué decir

«El servidor negocia de a un requisito y responde con códigos: el 403 era por el User-Agent, el 400 por un header y un parámetro que faltaban, el 406 por el idioma y el 401 por autenticación Basic. Fui agregando cada cosa sin sacar las anteriores. El nombre `tesoro.lab` solo resuelve del lado del proxy, por eso uso `socks5h` y no `socks5`.»

⚠ Dos detalles

`-A "Chrome"` no alcanza (el valor empieza con `Mozilla/5.0`); y la URL con `?` va entre comillas o la shell rompe el pedido.

4 nginx: servidor web y proxy reverso

La idea

nginx es un servidor HTTP. En los parciales lo usás de dos formas:

1. **Servidor de archivos (hosting)**: recibe `GET /` y devuelve un archivo del disco (`index.html`).
2. **Proxy reverso**: recibe el pedido y, en vez de buscar un archivo, se lo reenvía a otro servidor (`proxy_pass`) y le devuelve al cliente la respuesta. El cliente nunca se entera: la URL del navegador no cambia. Esto es lo que piden en «no se debe ver modificada la URL ingresada».

> 💡 **Virtual hosts: muchos sitios, una sola IP**
>
> Varios nombres (`foo`, `bar`, `itba.example.org`) pueden apuntar a la misma IP. ¿Cómo sabe nginx qué sitio servir? Por el header **`Host`** del request: cada bloque `server` tiene un `server_name` y nginx elige el que coincide. Si ninguno coincide (por ejemplo entrando por IP), usa el **`default_server`**. Es exactamente el ejercicio foo / bar / «What?».

Redirect vs proxy reverso

	Redirect (301/302)	Proxy reverso (proxy_pass)
Quién va a buscar el contenido	El cliente, haciendo un segundo pedido	nginx, por detrás
¿Cambia la URL del navegador?	Sí	No
Config nginx	`return 301 http://otro/;`	`proxy_pass http://otro;`

Por eso cuando el enunciado dice «la URL no debe cambiar» la respuesta es proxy reverso. Y si dice «la URL puede cambiar» (E48 de la guía), alcanza con un redirect.

Dónde está cada cosa

```
/etc/nginx/nginx.conf              # config principal; dentro del bloque http {} incluye a las
otras
/etc/nginx/sites-available/        # un archivo por sitio ("los que existen")
/etc/nginx/sites-enabled/          # links simbólicos a los de arriba ("los que están
activos"): nginx lee SOLO estos
/etc/nginx/sites-enabled/default   # el sitio que viene de fábrica: ¡ya es default_server en el
puerto 80!
/var/www/html/                     # carpeta de archivos del sitio default
/var/log/nginx/access.log          # cada pedido que llega
/var/log/nginx/error.log           # errores (acá aparece por qué falla un proxy_pass,
permisos, etc.)
```

Comandos

```
sudo apt install nginx
sudo nginx -t                       # valida TODA la config y dice archivo y línea del error. Siempre antes de reiniciar
sudo systemctl restart nginx        # o reload, para aplicar cambios
sudo ln -s /etc/nginx/sites-available/foo /etc/nginx/sites-enabled/foo   # habilitar el sitio foo
sudo rm /etc/nginx/sites-enabled/default                                 # deshabilitar el default (borra solo el link)
curl -v -H 'Host: foo' http://127.0.0.1/      # probar un virtual host SIN tocar DNS ni /etc/hosts
curl -v --resolve foo:80:127.0.0.1 http://foo/   # otra forma: curl hace de cuenta que foo = 127.0.0.1
```

Anatomía de un bloque server

```
server {
    listen 80;                          # puerto (y opcionalmente IP: listen 127.0.0.1:80;)
    server_name foo foo.example.org;    # nombres (header Host) que atiende este bloque
    root /var/www/foo;                  # carpeta de donde salen los archivos
    index index.html;                   # qué archivo servir cuando piden un directorio (/)

    access_log /var/log/nginx/foo_access.log;
    error_log  /var/log/nginx/foo_error.log;

    location / {                        # reglas para los paths que empiezan con /
        try_files $uri $uri/ =404;      # probá el archivo, después como directorio, si no 404
    }
}
```

Directiva	Qué hace
`listen 80 default_server;`	Este bloque atiende lo que no matchea ningún otro server_name en ese puerto. Solo puede haber uno por puerto
`server_name _;`	Un nombre «inválido» a propósito, usado por convención en el default
`return 200 "What?\n";`	Responde directo ese código y texto, sin archivos
`default_type text/plain;`	Content-Type para lo que devuelve `return` (si no, sale `application/octet-stream` y el navegador lo descarga)
`location = /x` · `location /x` · `location ~ \.php$`	Coincidencia exacta · por prefijo (gana el más largo) · por expresión regular
`proxy_pass http://host;`	Reenviar el pedido a otro servidor
`proxy_set_header H v;`	Setear un header en el pedido que nginx manda al servidor de atrás
`add_header H v;`	Agregar un header a la respuesta que recibe el cliente
`gzip on;`	Comprimir respuestas cuando el cliente manda `Accept-Encoding: gzip`

proxy_pass: dos detalles que importan

1. La barra final. Si `proxy_pass` tiene solo host (`http://foo.leak.com.ar`), el path del pedido se manda *tal cual*. Si tiene un path (aunque sea `/`), la parte que matcheó el `location` se *reemplaza* por ese path.

```
location / { proxy_pass http://foo.leak.com.ar; }       # /xyz/a ->
http://foo.leak.com.ar/xyz/a
location / { proxy_pass http://foo.leak.com.ar/; }      # /xyz/a ->
http://foo.leak.com.ar/xyz/a  (mismo resultado acá)
location /app/ { proxy_pass http://localhost:8080/; }   # /app/a -> http://localhost:8080/a
(se "come" el /app/)
location /app/ { proxy_pass http://localhost:8080; }    # /app/a -> http://localhost:8080/app/a
```

2. El header Host. Por defecto nginx manda al servidor de atrás `Host: <el host del proxy_pass>` (la variable `$proxy_host`). Es lo correcto cuando el de atrás es un servidor externo con virtual hosts (`foo.leak.com.ar`, `protos.foo`): él necesita ver su propio nombre para saber qué sitio servir. Si pusieras `proxy_set_header Host $host;` le llegaría `Host: foo.example.org`, no lo reconocería y te devolvería otro sitio o un error.

> **⚠ HTTPS hacia atrás**
>
> Si el `proxy_pass` es `https://...`, agregá `proxy_ssl_server_name on;`. Hace que nginx mande el nombre del servidor en el handshake TLS (SNI, capítulo 5). Sin eso, muchos servidores con varios dominios en la misma IP cortan el handshake o devuelven otro certificado.

Compresión transparente (gzip)

```
# dentro de http {} en nginx.conf (vienen comentadas: se descomentan) o dentro del server {}
gzip on;
gzip_types text/plain text/css application/json application/javascript text/xml
application/xml;
                                # text/html se comprime siempre; el resto hay que listarlo
gzip_proxied any;               # comprimir también si el pedido ya vino de otro proxy
(header Via)
gzip_vary on;                   # agrega "Vary: Accept-Encoding" para que los cachés no
mezclen versiones
# Con proxy reverso, para que comprima nginx y no el servidor de atrás:
#   proxy_set_header Accept-Encoding "";   (el de atrás manda sin comprimir, nginx comprime)
```

```
curl -s -o /dev/null -w '%{size_download}\n' http://foo.example.org/               # bytes
sin comprimir
curl -s -o /dev/null -w '%{size_download}\n' --compressed http://foo.example.org/   # menos
bytes
curl -v --compressed http://foo.example.org/ 2>&1 | grep -i content-encoding       # <
Content-Encoding: gzip
```

Es «transparente» porque el cliente pide `Accept-Encoding: gzip`, el servidor responde con `Content-Encoding: gzip` y el cliente descomprime solo: el usuario no ve la diferencia. Ventaja: menos bytes por la red (mejor en conexiones lentas). Costo: CPU en el servidor para comprimir y en el cliente para descomprimir. Para imágenes JPEG/PNG no conviene porque ya vienen comprimidas.

Ejercicios de parciales resueltos

Recuperatorio 1C2021 Ejercicio I (y E36 de la guía)

Hosting para `itba.example.org` y `www.example.org`. `/` de itba.example.org devuelve «Ad Lucem Serenitate Tendo»; `/` de www.example.org devuelve «El RFC 2606 le da un estado especial a example.org.»; cualquier otro dominio que apunte al servidor devuelve «What?». Debe funcionar desde Chrome/Firefox y demostrarse con curl.

```
# 1) Contenido de cada sitio
sudo mkdir -p /var/www/itba /var/www/www
echo 'Ad Lucem Serenitate Tendo' | sudo tee /var/www/itba/index.html
echo 'El RFC 2606 le da un estado especial a example.org.' | sudo tee /var/www/www/index.html
```

```
# 2) /etc/nginx/sites-available/parcial
server {
    listen 80;
    server_name itba.example.org;       # atiende cuando Host: itba.example.org
    root /var/www/itba;
    index index.html;
}
server {
    listen 80;
    server_name www.example.org;
    root /var/www/www;
    index index.html;
}
server {
    listen 80 default_server;           # todo lo que no matchee arriba (IP, otros nombres)
    server_name _;
    default_type text/plain;
    return 200 "What?\n";
}
```

```
# 3) Habilitar, sacar el default de fábrica (que también es default_server), validar y
reiniciar
sudo ln -s /etc/nginx/sites-available/parcial /etc/nginx/sites-enabled/
sudo rm /etc/nginx/sites-enabled/default
sudo nginx -t && sudo systemctl restart nginx

# 4) Que el navegador encuentre los nombres: /etc/hosts
127.0.0.1  itba.example.org www.example.org

# 5) Demostrar
curl http://itba.example.org/           # Ad Lucem Serenitate Tendo
curl http://www.example.org/            # El RFC 2606 ...
curl http://127.0.0.1/                  # What?   (Host: 127.0.0.1 no coincide con nada)
curl -H 'Host: cualquiercosa' http://127.0.0.1/   # What?
```

⚠ Trampas de este ejercicio

- Si no borrás `sites-enabled/default`, `nginx -t` se queja: *a duplicate default server for 0.0.0.0:80*.
- `www.example.org` es un dominio real. Sin la línea en `/etc/hosts`, curl iría al example.org de verdad.
- Con nombres sin punto (`foo`) Chrome puede tomarlo como búsqueda en Google: escribí `http://foo/` completo. Y ojo con TLDs como `.foo` o `.dev` : están en la lista de precarga HSTS de los navegadores, que fuerzan HTTPS aunque escribas http. Ese es el «problema al acceder http://foo/ desde un user agent avanzado» que menciona la guía.

🗣 Qué decir

«Los tres nombres apuntan a la misma IP. nginx distingue el sitio por el header Host del request y elige el bloque server cuyo server_name coincide. Si ninguno coincide, por ejemplo al entrar por IP, responde el bloque marcado como default_server, que devuelve What?. Para que mi navegador resuelva los nombres los agregué en /etc/hosts apuntando a mi máquina.»

El clásico (Parciales 2020, 2021, 2022, recus) — proxy reverso

Al ingresar a `http://foo.example.org/` con cualquier user agent se debe servir el contenido de `http://foo.leak.com.ar/`. No se debe ver modificada la URL. Si se usa curl no se permiten otras opciones. Se puede pedir cualquier recurso dentro de foo.leak.com.ar.
Variante 2021: el contenido se debe servir comprimido de forma transparente si el cliente lo pide.
Variante 2024-2C: el origen es `http://protos.sebikul.com/` y se debe conservar el valor del header `X-Parcial-Protos`.

```
# /etc/nginx/sites-available/foo.example.org
server {
    listen 80;
    server_name foo.example.org;

    location / {
        proxy_pass http://foo.leak.com.ar;   # sin path: /cualquier/cosa ->
foo.leak.com.ar/cualquier/cosa
        # Host queda en foo.leak.com.ar (default $proxy_host): el servidor de atrás reconoce su
sitio
    }

    # --- variante compresión transparente ---
    gzip on;
    gzip_types text/plain text/css application/javascript application/json;
    gzip_proxied any;
    proxy_set_header Accept-Encoding "";      # pido sin comprimir al de atrás; comprime nginx

    # --- variante header 2024-2C (nginx ya pasa los headers del cliente; esto lo hace
explícito) ---
    # proxy_set_header X-Parcial-Protos $http_x_parcial_protos;   # $http_<nombre> = header del
request
}
```

```
sudo ln -s /etc/nginx/sites-available/foo.example.org /etc/nginx/sites-enabled/
echo '127.0.0.1 foo.example.org' | sudo tee -a /etc/hosts     # -a: agregar al final, no pisar el archivo
sudo nginx -t && sudo systemctl restart nginx
curl -v http://foo.example.org/                # mismo contenido que foo.leak.com.ar, URL intacta
curl -v --compressed http://foo.example.org/    # (variante) muestra Content-Encoding: gzip
curl -i -H 'X-Parcial-Protos: pincha' http://foo.example.org/   # (variante) verificar con Wireshark que el header sale
```

🗣 Qué decir

«Configuré nginx como proxy reverso: el navegador resuelve foo.example.org a mi máquina por /etc/hosts, le pide a mi nginx, y nginx hace el pedido a foo.leak.com.ar y devuelve la respuesta. Como el que va a buscar el contenido es el servidor y no el cliente, la URL no cambia (con un redirect sí cambiaría). nginx manda Host: foo.leak.com.ar porque es lo que ese servidor espera para elegir su virtual host.»

⚠ Para el video

«Si para la resolución tuviese que configurar un servidor, inicie el video mostrando que el servidor no está corriendo»: arrancá con `sudo systemctl stop nginx`, mostrá que `curl` falla (Connection refused), después `start` y la demostración.

Ejercicios de repaso (Thomas Mizrahi)

(a) Sitio `puerro` como proxy reverso hacia `https://protos.foo/` (puerro/xyz/... debe ir a https://protos.foo/xyz/...); sitio `hola` con una página de bienvenida; cualquier otro sitio «what?». Acceder con solo `curl puerro`. (b) Que `curl 127.0.0.1` diga «hola, yo!» y `curl <ipv4 de la máquina>` diga «vos no sos yo».

```
# (a)
server {
    listen 80;
    server_name puerro;
    location / {
        proxy_pass https://protos.foo;      # sin path: conserva /xyz/...
        proxy_ssl_server_name on;           # manda SNI=protos.foo en el handshake TLS
    }
}
server {
    listen 80;
    server_name hola;
    root /var/www/hola;                     # con index.html: <h1>¡Bienvenido!</h1>
    index index.html;
}
server {
    listen 80 default_server;
    server_name _;
    default_type text/plain;
    return 200 "what?\n";
}
# /etc/hosts:  127.0.0.1 puerro hola

# (b) curl 127.0.0.1 manda Host: 127.0.0.1 ; curl 192.168.0.10 manda Host: 192.168.0.10
server {
    listen 80;
    server_name 127.0.0.1;
    default_type text/plain;
    return 200 "hola, yo!\n";
}
server {
    listen 80 default_server;
    server_name _;
    default_type text/plain;
    return 200 "vos no sos yo\n";
}
# Alternativa: distinguir por la IP local donde llegó la conexión: listen 127.0.0.1:80; vs
listen 192.168.0.10:80;
```

Guía E39 — proxy hacia un servidor local

Levantar `python3 -m http.server 8080` en `/tmp/protos` y que el host `foo` muestre ese contenido.

```
location / { proxy_pass http://127.0.0.1:8080; }
# Ventajas del esquema: nginx atiende muchas conexiones, comprime, cachea, termina TLS y
protege al
# servidor de aplicación (python, tomcat), que queda escuchando solo en localhost, sin
exponerse.
```

✍ Practicá en papel — Capítulo 4

1. Tenés tres server: `server_name a;`, `server_name b;` y uno con `listen 80 default_server`. ¿Qué bloque atiende `curl -H 'Host: c' http://127.0.0.1/`? ¿Y `curl http://a/` si `a` no está en /etc/hosts?
2. Con `location /api/ { proxy_pass http://127.0.0.1:3000/; }`, ¿a qué URL va `GET /api/users/7`? ¿Y si sacás la barra final?
3. ¿Por qué no conviene `proxy_set_header Host $host;` cuando el de atrás es `foo.leak.com.ar`?
4. Escribí de memoria un bloque server que devuelva «What?» a todo lo desconocido.
5. Te dicen «la URL puede cambiar». Escribí la solución más simple con nginx.
6. ¿Cómo demostrás con curl que la compresión funciona? ¿Qué header mirás?

5 Criptografía, TLS y certificados

Las cuatro piezas de la criptografía

Antes de TLS hay que tener claras cuatro herramientas. TLS (y SSH, capítulo 10) son combinaciones de estas piezas.

Pieza	Qué hace	Ejemplos	Analogía
Cifrado simétrico	La misma clave cifra y descifra. Rapidísimo. Problema: ¿cómo comparto la clave sin que la vean?	AES-128/256 (modo GCM), ChaCha20	Un candado con una llave que tienen los dos
Cifrado asimétrico	Par de claves: lo que cifra la **pública** solo lo descifra la **privada**. La pública se reparte, la privada no sale nunca de su dueño. Lento	RSA, curvas elípticas (ECDSA, Ed25519)	Un buzón: cualquiera mete cartas, solo el dueño abre
Hash	Resumen de tamaño fijo de cualquier dato. Mismo dato → mismo hash; un bit distinto → hash totalmente distinto. No se puede volver atrás	SHA-256, SHA-384 (MD5 y SHA-1 están rotos)	Una huella digital
Firma digital	Hash del mensaje cifrado con la **privada**. Cualquiera la verifica con la **pública**. Prueba quién lo firmó y que no se modificó	RSA-SHA256, ECDSA, Ed25519	Firma de escribano

Y una quinta, el **intercambio de claves Diffie-Hellman** (DHE, o ECDHE con curvas elípticas): dos partes que se hablan en público terminan con un secreto compartido que nadie que escuchó puede calcular. Es la forma moderna de resolver «¿cómo comparto la clave simétrica?».

> 💡 **Codificar ≠ cifrar ≠ hashear**
>
> **Codificar** (base64, quoted-printable, percent-encoding, gzip) es cambiar el formato: cualquiera lo revierte, no hay secreto. **Cifrar** necesita una clave para revertir. **Hashear** no se revierte. Si en el oral decís «la password va cifrada en base64» perdés puntos.

Certificados: ¿cómo sé que la clave pública es de quien dice ser?

Si un atacante en el medio te da *su* clave pública haciéndose pasar por el banco, cifrás todo para él. El certificado resuelve eso: es un documento que dice «la clave pública *X* pertenece a `www.banco.com`», **firmado** por una **Autoridad Certificante (CA)** en la que tu sistema ya confía.

Un certificado X.509 contiene, entre otras cosas:

- **Subject** (a quién pertenece) y sobre todo **Subject Alternative Name (SAN)**: la lista de nombres válidos (`DNS:seguro.protos.foo`). Los navegadores modernos miran el SAN, no el CN.
- **Issuer**: qué CA lo firmó.
- **Validez**: Not Before / Not After.
- La **clave pública** del dueño.
- La **firma** de la CA sobre todo lo anterior.

La cadena de confianza (PKI)

```
CA raíz  (autofirmada; viene instalada en el sistema/navegador: el "trust store")
   └── firma a ─► CA intermedia
                    └── firma a ─► certificado de www.ejemplo.com  (el que manda el servidor)
```

El cliente verifica firma por firma hasta llegar a una raíz que tiene en su trust store (en Debian/Ubuntu, `/etc/ssl/certs/`). Además verifica que el nombre al que te conectaste esté en el SAN y que no esté vencido. Si algo falla: *«NET::ERR_CERT_AUTHORITY_INVALID»* en Chrome, o en curl *«SSL certificate problem: unable to get local issuer certificate»*.

- **Certificado autofirmado**: se firma a sí mismo. Cifra igual de bien, pero nadie lo reconoce, así que el cliente da error salvo que lo agregues a mano al trust store.
- **mkcert** (lo que propone la guía, E41): crea una **CA local**, la instala en tu trust store (`mkcert -install`) y emite certificados firmados por ella. Resultado: en *tu* máquina, curl y el navegador confían sin warnings.
- En producción se usa una CA pública, típicamente Let's Encrypt (gratis, con `certbot`), que verifica que controlás el dominio.

TLS: el handshake explicado sin humo

TLS (el sucesor de SSL) es una capa entre TCP y la aplicación. HTTPS = HTTP dentro de TLS, por defecto en el puerto 443. Garantiza tres cosas: **confidencialidad** (nadie lee), **integridad** (nadie modifica sin que se note) y **autenticación del servidor** (hablás con quien creés).

Handshake de TLS 1.3, simplificado:

1. **ClientHello**: el cliente manda las versiones y cipher suites que soporta, un número aleatorio, su parte del intercambio Diffie-Hellman (key share) y el **SNI** (Server Name Indication: el nombre del sitio, en claro).
2. **ServerHello**: el servidor elige versión y cipher suite y manda su parte de Diffie-Hellman. Desde acá los dos ya calculan el mismo secreto y todo lo que sigue va cifrado.
3. El servidor manda su **Certificate** (y la cadena) y un **CertificateVerify**: una firma, hecha con su clave privada, sobre todo el handshake. Así prueba que tiene la privada que corresponde al certificado.
4. El cliente valida la cadena, el nombre y la firma. Los dos mandan **Finished**.
5. Los datos de la aplicación (el GET de HTTP) viajan cifrados con **cifrado simétrico** (AES-GCM o ChaCha20) usando claves derivadas del secreto compartido.

> 💡 **Por qué se combinan asimétrico y simétrico**
>
> El asimétrico (y Diffie-Hellman) es lento pero resuelve cómo acordar una clave con un desconocido y cómo probar identidad. El simétrico es rápido para cifrar megas de datos. TLS usa lo lento una vez, al principio, para acordar la clave de lo rápido.

> 💡 **Forward secrecy y por qué importa el SNI**
>
> Con (EC)DHE la clave de sesión se genera de cero en cada conexión y se tira. Si mañana roban la clave privada del servidor, no pueden descifrar el tráfico grabado hoy: eso es **forward secrecy**. El **SNI** existe porque en una misma IP puede haber muchos sitios HTTPS: el servidor necesita saber qué certificado mostrar *antes* de que llegue el header Host, que viaja cifrado.

Cómo leer una cipher suite

```
TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256          (formato TLS 1.2)
    └──┘ └┘      └─────────┘ └──┘
      │    │            │        └─ hash para derivar claves e integridad
      │    │            └─ cifrado simétrico de los datos (AES 128 bits, modo GCM =
autenticado)
      │    └─ con qué se firma / autentica el servidor (certificado RSA)
      └─ intercambio de claves (Diffie-Hellman efímero con curvas elípticas -> forward secrecy)

TLS_AES_256_GCM_SHA384                          (TLS 1.3: el intercambio es siempre (EC)DHE y
se negocia aparte)
```

Cosas que hoy se consideran inseguras: SSLv3, TLS 1.0 y 1.1, RC4, 3DES, MD5, SHA-1 en firmas, e intercambio por RSA estático (sin forward secrecy).

Herramientas

```
# Generar clave + certificado autofirmado (una línea, sin preguntas)
openssl req -x509 -newkey rsa:2048 -nodes -days 365 \
  -keyout seguro.key -out seguro.crt \
  -subj "/CN=seguro.protos.foo" -addext "subjectAltName=DNS:seguro.protos.foo"
#  req -x509  -> en vez de un pedido de firma (CSR), generá directamente un certificado
#  -newkey rsa:2048 -> creá una clave privada RSA nueva de 2048 bits
#  -nodes     -> no cifres la clave privada con passphrase (si no, nginx la pide al arrancar)
#  -subj / -addext -> nombre del sujeto y SAN, para que el certificado valga para ese dominio

openssl x509 -in seguro.crt -noout -text        # ver el contenido: Issuer, Validity, SAN,
clave pública
openssl s_client -connect protos.foo:443 -servername protos.foo   # handshake a mano: muestra
cadena, versión y suite
curl -v https://seguro.protos.foo/              # en -v se ven el handshake TLS, el certificado
y su verificación

# Con mkcert (CA local confiable)
mkcert -install                                 # crea la CA local y la instala en el trust
store del sistema
mkcert seguro.protos.foo                        # -> seguro.protos.foo.pem (cert) y
seguro.protos.foo-key.pem (privada)

# Confiar en un certificado propio a nivel sistema (Debian/Ubuntu) -> curl deja de quejarse sin
-k
sudo cp seguro.crt /usr/local/share/ca-certificates/seguro.crt     # tiene que terminar en .crt
sudo update-ca-certificates
```

nginx con HTTPS

```
server {
    listen 443 ssl;                                      # TLS en el 443
    server_name seguro.protos.foo;
    ssl_certificate     /etc/nginx/ssl/seguro.crt;       # parte PÚBLICA (certificado, + cadena si
hay intermedias)
    ssl_certificate_key /etc/nginx/ssl/seguro.key;       # parte PRIVADA: permisos restringidos,
no se comparte
    ssl_protocols TLSv1.2 TLSv1.3;                       # solo versiones seguras
    location / { ... }
}
server {                                                 # opcional: forzar HTTPS
    listen 80;
    server_name seguro.protos.foo;
    return 301 https://$host$request_uri;                # redirect permanente a la versión segura
}
```

Ejercicio de parcial resuelto

Parcial 2Q2025 — Ejercicio IV

El servidor en `parcial.protos.foo` está corriendo un servidor HTTP inseguro. Como administrador responsable desea exponerlo al público a través de HTTPS. Configure lo necesario para que al acceder a `https://seguro.protos.foo` con un HTTP User Agent (Firefox, Chrome, curl) se sirva el contenido devuelto de `parcial.protos.foo`. Requisito: no se debe ver modificada la URL; si se usa curl no se permiten otras opciones (`$ curl https://seguro.protos.foo`).

Es el proxy reverso del capítulo 4 más TLS. El detalle fino: como **no podés usar** `-k`, curl tiene que **confiar** en tu certificado. Por eso no alcanza con uno autofirmado suelto: o usás mkcert (CA local instalada) o agregás tu certificado al trust store del sistema.

```
# 1) Nombre -> mi máquina
echo '127.0.0.1 seguro.protos.foo' | sudo tee -a /etc/hosts

# 2) Certificado confiable para ese nombre
mkcert -install
mkcert seguro.protos.foo
sudo mkdir -p /etc/nginx/ssl && sudo mv seguro.protos.foo*.pem /etc/nginx/ssl/

# 3) /etc/nginx/sites-available/seguro
server {
    listen 443 ssl;
    server_name seguro.protos.foo;
    ssl_certificate     /etc/nginx/ssl/seguro.protos.foo.pem;
    ssl_certificate_key /etc/nginx/ssl/seguro.protos.foo-key.pem;

    location / {
        proxy_pass http://parcial.protos.foo;   # el backend inseguro; Host: parcial.protos.foo
    }
}

# 4) Activar y probar
sudo ln -s /etc/nginx/sites-available/seguro /etc/nginx/sites-enabled/
sudo nginx -t && sudo systemctl restart nginx
curl https://seguro.protos.foo              # contenido de parcial.protos.foo, sin errores de
certificado
curl -v https://seguro.protos.foo 2>&1 | grep -iE 'SSL connection|subject|issuer|verify'
```

⚠ Firefox

Firefox tiene su propio trust store. mkcert lo cubre si está instalado `certutil` (paquete `libnss3-tools`) *antes* de correr `mkcert -install`. Si no, Firefox muestra la advertencia aunque curl ande.

🗣 Qué decir

«nginx termina TLS: el cliente negocia HTTPS con mi nginx, que presenta un certificado para seguro.protos.foo firmado por una CA local que instalé en el trust store con mkcert. Por eso curl lo valida sin -k. Una vez descifrado el pedido, nginx actúa de proxy reverso hacia parcial.protos.foo por HTTP, así que la URL del cliente no cambia. El tramo inseguro queda entre mi proxy y el backend, no expuesto al usuario.»

✍ Practicá en papel — Capítulo 5

1. ¿Qué clave usa el servidor para firmar el CertificateVerify? ¿Con qué la verifica el cliente?
2. Explicá por qué TLS no cifra todo con RSA directamente.
3. ¿Qué significa cada parte de `TLS_ECDHE_ECDSA_WITH_AES_256_GCM_SHA384`?
4. Tu certificado dice `CN=foo` sin SAN y Chrome lo rechaza aunque la CA sea confiable. ¿Por qué?
5. ¿Por qué el SNI viaja sin cifrar en el ClientHello?
6. ¿Qué diferencia hay entre `ssl_certificate` y `ssl_certificate_key`? ¿Cuál nunca se comparte?
7. ¿Por qué en el 2Q2025 no alcanza con un autofirmado y `curl -k`?

6 DNS: cómo funciona y cómo consultarlo

La idea

DNS es la «guía telefónica» distribuida de Internet: traduce nombres a datos (IPs, servidores de mail, textos). Nadie tiene toda la guía: está repartida en un **árbol** donde cada nodo delega en sus hijos.

```
                      .   (raíz: 13 nombres de root servers, a.root-servers.net ... m.)
          ┌───────────┼──────────┐
        com         ar         org          <- TLDs (dominios de primer nivel)
         │          │
      google       edu
                    │
                   itba                     <- itba.edu.ar
                    │
                    it                      <- it.itba.edu.ar (una zona aparte, delegada)
                    │
                 pampero                    <- pampero.it.itba.edu.ar.  (FQDN: termina en el
punto de la raíz)
```

Los actores

- **Servidor autoritativo**: tiene los datos «oficiales» de una **zona** (lo que configurás con BIND). Contesta con el flag `aa` (authoritative answer).
- **Resolver recursivo** (el DNS de tu ISP, 8.8.8.8, el de la facultad): hace todo el trabajo de recorrer el árbol por vos y guarda las respuestas en **caché** durante el TTL.
- **Stub resolver**: la parte de tu sistema operativo que le pregunta al recursivo (configurado en `/etc/resolv.conf`). Antes mira `/etc/hosts`.

Una resolución completa

1. Tu máquina le pregunta al recursivo: «¿A de pampero.it.itba.edu.ar?» (consulta **recursiva**: «resolvémelo todo»).
2. El recursivo, con caché vacía, le pregunta a un **root**: no sabe, pero responde «andá a los servidores de `ar.`» (un **referral**: registros NS en la sección AUTHORITY).
3. Pregunta a `ar.` → «andá a los de `edu.ar.`» → «andá a los de `itba.edu.ar.`» → «andá a los de `it.itba.edu.ar.`». Estas son consultas **iterativas**.
4. El autoritativo de `it.itba.edu.ar.` responde la IP con el flag `aa`.
5. El recursivo la guarda en caché por el TTL y te la devuelve.

`dig +trace` hace exactamente este recorrido desde la raíz y te muestra cada salto (E44 y E45 de la guía).

> 💡 **Glue records**
> Si `itba.edu.ar` dice «mi servidor de nombres es `ns1.itba.edu.ar`», ¿cómo resolvés `ns1.itba.edu.ar` si para eso necesitás preguntarle a él? El padre (`edu.ar`) incluye la IP de ese NS en la sección ADDITIONAL. Esos registros A «de ayuda» se llaman **glue**.

Tipos de registro (Resource Records)

Tipo	Qué guarda	Ejemplo en un archivo de zona
A	Nombre → IPv4	`pampero IN A 192.168.70.10`
AAAA	Nombre → IPv6	`www IN AAAA 2001:db8::1`
CNAME	Alias → nombre canónico. «Este nombre es otro nombre, preguntá por aquel»	`correo IN CNAME pampero`
MX	Servidores que reciben el mail del dominio, con **preferencia** (menor número = se intenta primero)	`@ IN MX 10 mail.ejemplo.com.`
NS	Servidores autoritativos de la zona (o de una subzona delegada)	`@ IN NS ns1.ejemplo.com.`
SOA	Start of Authority: datos administrativos de la zona. Uno solo, al principio	ver capítulo 7
TXT	Texto libre. Se usa para SPF, DKIM, DMARC y verificaciones de dominio	`@ IN TXT "v=spf1 ip4:1.2.3.4 -all"`
PTR	IP → nombre (resolución inversa), en la zona `in-addr.arpa`	`10 IN PTR pampero.it.itba.edu.ar.`
SRV, CAA	Ubicación de servicios (host+puerto) · qué CAs pueden emitir certificados para el dominio	

⚠ Reglas de CNAME y MX (las toman)

- Un nombre que tiene CNAME **no puede tener ningún otro registro**. Por eso no podés poner un CNAME en el apex (`@`) de la zona: ahí siempre hay SOA y NS.
- Un **MX y un NS deben apuntar a un nombre que tenga registro A/AAAA**, nunca a un CNAME ni a una IP.
- Un registro **A** lleva una **IP**, nunca un nombre: `www IN A pampero` está mal.

¿Por qué usar CNAME en vez de una IP?

Si tu sitio está hosteado en un proveedor (Amazon S3/CloudFront, GitHub Pages, Heroku), apuntás `www IN CNAME algo.proveedor.com.`. El proveedor puede cambiar sus IPs, balancear entre muchas o elegir la más cercana al usuario, y vos no tenés que tocar tu zona nunca. Es la respuesta al ejercicio de repaso «investigue la cadena de CNAME de foo.leak.com.ar»: `dig foo.leak.com.ar` muestra en ANSWER cada CNAME de la cadena hasta el A final, y la IP te dice dónde está hosteado (en un apunte de 2022, `foo.leak.com.ar` resolvía a `52.92.x.x`, rango de Amazon S3).

TTL y caché

Cada registro tiene un **TTL** en segundos: cuánto puede guardarlo en caché un resolver. TTL alto = menos consultas pero los cambios tardan en verse. Antes de una migración se baja el TTL. También existe la **caché negativa**: si un nombre no existe (NXDOMAIN), los resolvers recuerdan «no existe» durante el tiempo que dice el último campo del SOA.

dig: la herramienta principal

```
dig pampero.it.itba.edu.ar            # consulta A (por defecto) al DNS configurado en el
sistema
dig +short pampero.it.itba.edu.ar     # solo la respuesta, sin todo el detalle
dig MX itba.edu.ar                    # tipo de registro: MX, NS, TXT, SOA, AAAA, CNAME, ANY...
dig TXT proto.leak.com.ar +short      # para ver SPF
dig @8.8.8.8 itba.edu.ar              # @servidor: preguntarle a ESE servidor (ej: @127.0.0.1 a
tu BIND)
dig NS it.itba.edu.ar                 # quiénes son los autoritativos de la zona
dig SOA itba.edu.ar                   # el SOA: servidor primario, mail del responsable,
serial, tiempos
dig -x 10.16.1.102                    # resolución inversa (PTR) de una IP
dig +trace pampero.it.itba.edu.ar     # recorre desde la raíz, mostrando cada delegación
dig +norecurse @a.root-servers.net ar NS   # una consulta iterativa a mano
host pampero.it.itba.edu.ar           # versión resumida
nslookup -type=MX itba.edu.ar         # la herramienta vieja (también existe en Windows)
whois itba.edu.ar                     # quién registró el dominio (no es DNS: es el registro de
dominios)
```

Cómo leer la salida de dig

```
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 4417   ; NOERROR | NXDOMAIN (no existe) |
SERVFAIL (el servidor falló)
;; flags: qr aa rd ra; QUERY: 1, ANSWER: 2, AUTHORITY: 1, ADDITIONAL: 2
;;   qr = es respuesta | aa = respuesta AUTORITATIVA | rd = pedí recursión | ra = el servidor
ofrece recursión

;; QUESTION SECTION:
;demiloo.com.ar.            IN  MX                      ; lo que pregunté

;; ANSWER SECTION:
demiloo.com.ar.   604800    IN  MX  20 smtp20.proto.leak.com.ar.   ; nombre  TTL  clase  tipo
datos
demiloo.com.ar.   604800    IN  MX  30 smtp30.proto.leak.com.ar.

;; AUTHORITY SECTION:
demiloo.com.ar.   86400     IN  NS  ns1.demiloo.com.ar.   ; quiénes son autoridad

;; ADDITIONAL SECTION:
ns1.demiloo.com.ar. 86400   IN  A   192.168.1.50          ; datos de ayuda (glue)

;; SERVER: 127.0.0.1#53(127.0.0.1)                         ; quién me respondió
```

Ejercicios de parciales resueltos

Recuperatorio — Ejercicio I

Sobre el dominio `proto.leak.com.ar`: A. ¿Cuáles son las direcciones IP de los servidores SMTP que reciben el correo entrante? B. Según la política SPF, ¿cuáles son las IPs que tienen permitido enviar correos salientes?

```
dig MX proto.leak.com.ar +short
#   20 smtp20.proto.leak.com.ar.      <- el correo entrante va a estos servidores, primero al
de menor preferencia
#   30 smtp30.proto.leak.com.ar.
dig +short smtp20.proto.leak.com.ar   # 127.0.0.1
dig +short smtp30.proto.leak.com.ar   # 192.168.122.235
dig TXT proto.leak.com.ar +short
#   "v=spf1 ip4:192.168.122.235 -all"  <- solo esa IP puede enviar; -all = cualquier otra,
rechazar
# A: 127.0.0.1 y 192.168.122.235   B: 192.168.122.235
```

(Valores que obtuvo un alumno en su momento; en el examen siempre consultá en vivo.)

🗣 Qué decir

«El correo entrante de un dominio lo indican sus registros MX; se ordenan por preferencia y el menor número se intenta primero. Como el MX da nombres, los resuelvo con consultas A. Para el saliente, la política SPF se publica en un registro TXT: el mecanismo ip4 enumera las IPs autorizadas y -all indica que cualquier otra debe rechazarse.»

Guía E42–E44

Autoridad de `google.com.`, `itba.edu.ar.`, `pampero.it.itba.edu.ar.`; servidores de correo entrante; todos los NS desde un root.

```
dig NS google.com +short              # autoritativos de la zona google.com
dig SOA pampero.it.itba.edu.ar        # para un host: la AUTHORITY SECTION muestra el SOA de la
zona que lo contiene
dig MX it.itba.edu.ar +short
dig +trace pampero.it.itba.edu.ar     # root -> ar. -> edu.ar. -> itba.edu.ar. ->
it.itba.edu.ar.
```

✍ Practicá en papel — Capítulo 6

1. Una respuesta de dig tiene flags `qr rd ra` pero no `aa`. ¿Qué te dice eso?
2. Un dominio tiene `MX 10 a.mail.com.` y `MX 5 b.mail.com.`. ¿A cuál se intenta entregar primero?
3. ¿Por qué `@ IN CNAME otro.com.` es inválido?
4. Dibujá la secuencia de consultas para resolver `www.fi.uba.ar` con la caché vacía. ¿Cuáles son recursivas y cuáles iterativas?
5. Explicá la diferencia entre `dig foo` y `dig @127.0.0.1 foo`.
6. Cambiaste la IP de un registro con TTL 86400 hace 10 minutos y tus usuarios siguen viendo la vieja. ¿Por qué? ¿Qué tendrías que haber hecho?

AMPLIACIÓN DEL CAPÍTULO 6 · DNS

Qué consultar según lo que dice el enunciado

El enunciado casi nunca te dice "corré tal comando": te describe **qué dato necesitás**. La traducción de esa frase al `dig` correcto es lo que se practica acá. La regla base: `dig [@servidor] [-p puerto] NOMBRE [TIPO] [+short]`.

LO QUE DICE EL ENUNCIADO	QUÉ PEDÍS
"la dirección IP de *nombre*"	`dig +short nombre` (tipo A, es el default)
"la IPv6 de *nombre*"	`dig +short nombre AAAA`
"los servidores que reciben el correo"	`dig MX dominio +short` → después un `dig +short` de cada nombre para su IP
"está publicado / documentado un texto, un token, una clave"	`dig TXT nombre +short`
"la política SPF / de envío"	`dig TXT dominio +short` (buscás `v=spf1 …`)
"quién es autoridad de la zona"	`dig NS zona +short`
"datos administrativos / de qué zona depende un host"	`dig SOA nombre` (mirás la AUTHORITY)
"el nombre de esta IP" (inverso)	`dig -x 10.0.0.5 +short` (registro PTR)
"consultando el servidor DNS de referencia en IP:puerto"	`dig @IP -p PUERTO nombre TIPO`
"recorré la resolución desde la raíz"	`dig +trace nombre`

💡 El parámetro que más se olvida: `-p`

Por defecto `dig` consulta al **DNS de tu sistema** en el puerto 53. Si el enunciado da un servidor de referencia (y más si usa un puerto no estándar), tenés que apuntar a él: `@IP` elige el servidor y `-p PUERTO` el puerto. Sin esto, preguntás al DNS equivocado y te da `NXDOMAIN` para un nombre que sí existe en el del lab.

```
dig @127.0.0.1 -p 5360 llave.internal.lab        # al DNS del lab, puerto 5360, no al del sistema
```

A vs TXT: la misma pregunta, distinto tipo

Elegir mal el tipo es el error más común. Guía rápida:

- Si el dato que buscás es **una dirección IP** (aunque después se use como otra cosa, por ejemplo una passphrase), es un registro **A**: `dig +short nombre`.
- Si el dato es **texto** (un token, una clave, una política, una pista), es un registro **TXT**: `dig TXT nombre +short`. El tipo A no te lo va a mostrar.

⚠ Los TXT vienen entre comillas

Un `dig TXT … +short` devuelve `"zorro-plateado"` con comillas. Antes de usar ese valor en otro comando, sacalas con `tr -d '"'`:

```
T=$(dig +short TXT token.internal.lab @127.0.0.1 -p 5360 | tr -d '"')
curl -s "http://127.0.0.1:8061/clave?token=$T"      # encadeno: el dato del DNS va directo al curl
```

🗣 Qué decir

«Según el dato que pide el enunciado elijo el tipo de registro: A para una IP, MX para los servidores de correo, TXT para texto como un token o la política SPF. Si hay un servidor de referencia, apunto con `@` y, si no es el puerto 53, con `-p`. Con `+short` me quedo solo con el dato para encadenarlo con el comando siguiente.»

7 BIND9: ser autoridad de una zona

Archivos involucrados

```
sudo apt install bind9 bind9-utils dnsutils
/etc/bind/named.conf                  # principal: incluye a los otros tres
/etc/bind/named.conf.options          # opciones globales: forwarders, recursion, listen-on
/etc/bind/named.conf.local            # ACÁ declarás tus zonas
/etc/bind/db.ejemplo.com              # el archivo de zona (el nombre lo elegís vos)
# en Fedora/CentOS: /etc/named.conf y /var/named/
```

1. Declarar la zona (named.conf.local)

```
zone "demiloo.com.ar" {                     // nombre de la zona (sin punto final acá)
    type master;                            // soy el primario: tengo el archivo original
("primary" en versiones nuevas)
    file "/etc/bind/db.demiloo.com.ar";     // dónde están los registros
};

zone "1.168.192.in-addr.arpa" {             // zona inversa para 192.168.1.0/24 (octetos al
revés)
    type master;
    file "/etc/bind/db.192.168.1";
};
```

2. El archivo de zona

```
$TTL 1d                                 ; TTL por defecto para registros que no indiquen uno
$ORIGIN demiloo.com.ar.                 ; (opcional) dominio que se agrega a los nombres
relativos
@   IN  SOA  ns1.demiloo.com.ar. demiloo.leak.com.ar. (
            2025102501   ; Serial: versión de la zona. AAAAMMDDnn. SIEMPRE subirlo al editar
            1h           ; Refresh: cada cuánto los secundarios preguntan si cambió el serial
            15m          ; Retry: si el refresh falló, reintentar cada...
            1w           ; Expire: si el primario no responde por este tiempo, el secundario
deja de responder la zona
            5m )         ; Negative TTL: cuánto se cachea un "no existe" (NXDOMAIN)

@        IN  NS     ns1.demiloo.com.ar.     ; quién es autoridad
ns1      IN  A      192.168.1.50            ; y su IP (el NS tiene que resolver a algo)
@    1h  IN  A      76.76.21.241            ; TTL propio de 1 hora para este registro
www  2w  IN  CNAME  demiloo.com.ar.         ; alias a la raíz de la zona, válido 2 semanas
@    1w  IN  MX 20  smtp20.proto.leak.com.ar.
@    1w  IN  MX 30  smtp30.proto.leak.com.ar.
```

Las reglas de sintaxis que más se rompen

Regla	Bien	Mal (y qué pasa)
Punto final: un nombre sin punto final es *relativo* y BIND le agrega el origen	`ns1.it.itba.edu.ar.` o solo `ns1`	`ns1.it.itba.edu.ar` → se convierte en `ns1.it.itba.edu.ar.it.itba.edu.ar.`
@ significa «el origen» (el nombre de la zona)	`@ IN MX 10 mail`	
Mail del responsable en el SOA: la `@` se escribe como punto	`admin.it.itba.edu.ar.`	`admin@it.itba.edu.ar.` (error de sintaxis)
Si el usuario del mail tiene un punto, se escapa	`juan\.perez.ejemplo.com.`	`juan.perez.ejemplo.com.` = juan@perez.ejemplo.com
Un A lleva una IP	`www IN CNAME pampero`	`www IN A pampero`
MX/NS apuntan a nombres con A propio	`@ IN MX 10 mail` + `mail IN A …`	MX a un nombre que no existe o a un CNAME
Línea que empieza con espacio = mismo nombre que la línea anterior	`@ IN MX 10 a.` `  IN MX 20 b.`	Sangrar por accidente cambia a qué nombre pertenece el registro
Unidades de TTL: `s m h d w`	`2w`, `1d`, `3600`	

3. Validar, recargar y probar

```
sudo named-checkconf                                          # sintaxis de named.conf*: sin salida = OK
sudo named-checkzone demiloo.com.ar /etc/bind/db.demiloo.com.ar   # valida la zona: "OK" y el serial cargado
sudo systemctl restart bind9        # (en algunas distros el servicio se llama named)
sudo rndc reload demiloo.com.ar     # recargar solo esa zona sin reiniciar
journalctl -u named -e              # (o -u bind9) si no carga: dice "zone ... not loaded due to errors"
dig @127.0.0.1 demiloo.com.ar SOA   # le pregunto A MI servidor: tiene que venir flag aa
dig @127.0.0.1 demiloo.com.ar A
dig @127.0.0.1 www.demiloo.com.ar   # muestra el CNAME y el A al que lleva
dig @127.0.0.1 demiloo.com.ar MX
dig @127.0.0.1 -x 192.168.1.50      # si configuraste la inversa
```

💡 Por qué el serial es tan importante

Los servidores secundarios comparan el serial del SOA con el que tienen. Si editás la zona y no lo subís, los secundarios creen que no hubo cambios y siguen sirviendo los datos viejos. Convención: `AAAAMMDDnn` (fecha + número de cambio del día), por ejemplo `2025103101`. Tiene que ser siempre **mayor** que el anterior.

Forwarders y recursión (named.conf.options)

```
options {
    directory "/var/cache/bind";
    recursion yes;                          // resolver también nombres de otras zonas para los clientes
    allow-recursion { 127.0.0.1; 192.168.1.0/24; };   // pero solo para mi red (si no, sos un "open resolver")
    forwarders { 8.8.8.8; 8.8.4.4; };       // lo que no sé, se lo pregunto a estos en vez de ir a la raíz
    listen-on { any; };
};
```

Sin forwarders, tu BIND resuelve iterando desde los root servers (vas a ver en Wireshark muchas consultas a distintos servidores). Con forwarders, manda una sola consulta recursiva a 8.8.8.8 (E54 de la guía). Para que tu propia máquina use tu BIND (E53), poné `nameserver 127.0.0.1` en `/etc/resolv.conf` (o configuralo en systemd-resolved).

Ejercicios de parciales resueltos

Parcial 2022 — Ejercicio 2 (demiloo.com.ar)

Instale BIND y sirva la zona `demiloo.com.ar`: 1) su servidor es autoridad; 2) contacto `demiloo@leak.com.ar`; 3) `demiloo.com.ar` tiene un A a la IP de `foo.leak.com.ar`, validez 1 hora; 4) `www` es CNAME a la raíz de la zona, validez 2 semanas; 5) recibe correo con los mismos servidores que `proto.leak.com.ar` (manteniendo el orden), validez 1 semana. Demostrar con dig.

```
dig +short foo.leak.com.ar          # la IP para el registro A (si es una cadena de CNAME, uso el A final)
dig MX proto.leak.com.ar +short     # copio nombres Y preferencias, para mantener el orden
```

El archivo de zona es el del ejemplo de arriba, con estos puntos justificados:

- **Autoridad**: SOA + NS apuntando a un nombre de mi zona (`ns1`) con su A a la IP de mi VM.
- **Contacto**: `demiloo@leak.com.ar` se escribe `demiloo.leak.com.ar.` en el SOA.
- **TTLs**: 1h, 2w y 1w se ponen en la columna de TTL de cada registro. Los dos MX tienen el mismo TTL porque forman un RRset (mismo nombre y tipo): el RFC exige que tengan el mismo.
- **Orden**: lo da la preferencia del MX, no el orden de las líneas. Por eso copio los mismos números.

```
dig @127.0.0.1 demiloo.com.ar A       # 3600 IN A ... con flag aa
dig @127.0.0.1 www.demiloo.com.ar     # www 1209600 IN CNAME demiloo.com.ar. + el A
dig @127.0.0.1 demiloo.com.ar MX      # 604800 IN MX 20 ... / 30 ...
dig @127.0.0.1 demiloo.com.ar SOA     # muestra el responsable demiloo.leak.com.ar.
```

Ejercicios de repaso — carlitos.com.ar

Autoridad de la zona; responsable `carloselmascapo@carlitos.com.ar`; `carlitos.com.ar` con A a 1.2.3.4 y TTL de 2 horas; `mysite` CNAME a `example.org` por una semana; correo en `mail20`, `mail30` y `mail40` con métricas 20, 30 y 40, validez 1 día.

```
$TTL 1d
@   IN SOA ns1.carlitos.com.ar. carloselmascapo.carlitos.com.ar. (
       2025102501 1h 15m 1w 5m )
@         IN NS     ns1.carlitos.com.ar.
ns1       IN A      192.168.1.50          ; IP de mi servidor
@     2h  IN A      1.2.3.4
mysite 1w IN CNAME  example.org.          ; punto final: es un nombre FUERA de mi zona
@     1d  IN MX 20  mail20
@     1d  IN MX 30  mail30
@     1d  IN MX 40  mail40
mail20    IN A      1.2.3.20              ; los MX tienen que resolver a algo (IPs de ejemplo)
mail30    IN A      1.2.3.30
mail40    IN A      1.2.3.40
```

Parcial 2Q2025 — Ejercicio III (debugging)

Administra BIND9 autoritativo para `it.itba.edu.ar`. Tras unos cambios, la resolución de todos los hosts de la zona dejó de funcionar. BIND se queja de múltiples errores al cargar `/etc/bind/db.it.itba.edu.ar`. Identifique y corrija todos los errores de sintaxis y de configuración lógica, y haga los cambios necesarios en el SOA para que los cambios se propaguen de manera correcta y rápida.

```
$TTL 604800
@ IN SOA ns1.it.itba.edu.ar. admin@it.itba.edu.ar. (    ; ✗ 1
        2025060500      ; Serial                        ; ✗ 6
        604800          ; Refresh                       ; ✗ 7
        86400           ; Retry
        2419200         ; Expire
        604800 )        ; Negative Cache TTL            ; ✗ 7
@       IN NS  ns1.it.itba.edu.ar.
@       IN MX  10 mail                                  ; ✗ 4
ns1.it.itba.edu.ar  IN A 192.168.70.1                   ; ✗ 2
pampero IN A 192.168.70.10
www     IN A pampero                                    ; ✗ 3
correo  IN CNAME pampero                                ; (válido, pero ojo: no usarlo como
destino del MX) ✗ 5
```

#	Error	Corrección y por qué
1	Mail del responsable con `@`	`admin.it.itba.edu.ar.`: en el SOA el mail se escribe como nombre de dominio (la `@` es un carácter especial: significa el origen)
2	`ns1.it.itba.edu.ar` sin punto final	Queda `ns1.it.itba.edu.ar.it.itba.edu.ar.`, entonces el NS `ns1.it.itba.edu.ar.` no tiene A: la zona no tiene un NS resoluble. Corregir a `ns1.it.itba.edu.ar.` o simplemente `ns1`
3	`www IN A pampero`	Un A lleva una IP. Para que www sea alias: `www IN CNAME pampero` (o `www IN A 192.168.70.10`)
4	`MX 10 mail` pero `mail` no existe	Agregar `mail IN A 192.168.70.10` (o la IP del servidor de correo), o apuntar el MX a `pampero`
5	Tentación: `MX 10 correo`	No: `correo` es CNAME y un MX no puede apuntar a un CNAME
6	Serial sin cambiar	Subirlo, por ejemplo `2025103101` (fecha del cambio + 01). Si no, los secundarios no se enteran
7	Tiempos demasiado largos	Refresh de 1 semana y negative TTL de 1 semana hacen que la corrección tarde días en propagarse (los resolvers recuerdan el «no existe» una semana). Para propagar rápido: refresh `1h`, retry `15m`, negative `5m`, y bajar `$TTL` (por ejemplo a `1h`)

```
$TTL 1h
@ IN SOA ns1.it.itba.edu.ar. admin.it.itba.edu.ar. (
        2025103101 ; Serial (subido)
        1h         ; Refresh
        15m        ; Retry
        2419200    ; Expire (4 semanas, está bien)
        5m )       ; Negative Cache TTL
@       IN NS     ns1.it.itba.edu.ar.
@       IN MX 10  mail
ns1     IN A      192.168.70.1
pampero IN A      192.168.70.10
mail    IN A      192.168.70.10
www     IN CNAME  pampero
correo  IN CNAME  pampero
```

```
sudo named-checkzone it.itba.edu.ar /etc/bind/db.it.itba.edu.ar   # antes: lista los errores; después: OK
sudo rndc reload it.itba.edu.ar
dig @127.0.0.1 pampero.it.itba.edu.ar +short
dig @127.0.0.1 www.it.itba.edu.ar
dig @127.0.0.1 it.itba.edu.ar MX
```

🗣 Qué decir

«Primero usé named-checkzone para que BIND me diga en qué línea falla. Hay errores de sintaxis, como la arroba en el mail del SOA y un nombre sin punto final que BIND completa con el origen, y errores lógicos: un registro A que apunta a un nombre en vez de a una IP y un MX hacia un host inexistente. Finalmente subí el serial, porque los secundarios solo transfieren la zona si el serial aumenta, y bajé el refresh y el TTL negativo para que la corrección se propague rápido y los resolvers no sigan cacheando la falla durante una semana.»

✍ Practicá en papel — Capítulo 7

1. Escribí de memoria una zona `pdc.lab` con: autoridad propia, responsable `root@pdc.lab`, `foo` → 10.0.0.5 (TTL 10 minutos), `bar` alias de foo, correo en `mx.pdc.lab` con preferencia 10.
2. En una zona con `$ORIGIN ejemplo.com.`, ¿cuál es el FQDN de cada uno? (a) `www` (b) `www.ejemplo.com` (c) `www.ejemplo.com.` (d) `@`
3. Escribí la declaración en `named.conf.local` y un PTR para que `dig -x 10.0.0.5` devuelva `foo.pdc.lab.`
4. Encontrá los errores: `@ IN MX mail.pdc.lab.` · `ftp IN CNAME 10.0.0.9` · `@ IN CNAME foo`.
5. ¿Qué comando te dice si la zona tiene errores sin reiniciar BIND?

8 Correo: SMTP, POP3, MIME y antispam

La idea: el correo es como el correo postal

```
  MUA (Gmail, Thunderbird)              MUA del destinatario
     | SMTP submission 587 (con usuario y clave)        ▲ POP3 110 / IMAP 143 (leer)
     ▼                                                  |
  MTA de tu proveedor ──SMTP 25──► MTA del destino (el MX del dominio) ──► MDA ──► casilla
(Maildir)
                    busca con DNS el MX del dominio destino
```

- **MUA** (Mail User Agent): el programa del usuario.
- **MTA** (Mail Transfer Agent): los servidores que se pasan el mail (Postfix, Exim, el de Google). Para entregar un mail a `@itba.edu.ar`, el MTA consulta los **MX** de ese dominio y prueba en orden de preferencia.
- **MDA**: deposita el mail en la casilla. El usuario lo lee con **POP3** (baja y suele borrar del servidor) o **IMAP** (deja todo en el servidor y sincroniza carpetas). En la guía: Postfix como MTA y Dovecot para POP3/IMAP.

Sobre vs carta: la distinción clave

> 💡 **Envelope (sobre) vs headers (carta)**
> **El sobre** son los comandos SMTP `MAIL FROM:` y `RCPT TO:`: es lo que los servidores usan para entregar y para devolver rebotes. **La carta** es lo que va dentro de `DATA`: los headers `From:`, `To:`, `Subject:` y el cuerpo. Es lo que el MUA *muestra*.
> Son independientes: el mail se entrega a quien dice `RCPT TO` aunque el `To:` diga otra cosa; el usuario ve como remitente el `From:` aunque el `MAIL FROM` sea otro; los rebotes van al `MAIL FROM`, que el servidor final guarda en el header `Return-Path:`. SMTP por sí solo no verifica ninguna correspondencia entre sobre y carta: por eso existen SPF, DKIM y DMARC.

Esto explica los incisos del E57 de la guía: un `From:` distinto en DATA cambia lo que muestra el MUA; un `To:` distinto en DATA no cambia a quién llega (manda el RCPT TO); y un RCPT TO inexistente genera un rebote a la dirección del MAIL FROM.

El protocolo SMTP, paso a paso

SMTP es un diálogo de texto: el cliente manda comandos de 4 letras y el servidor responde con un código de 3 dígitos (2xx ok, 3xx seguí, 4xx error temporal, 5xx error permanente). Se puede «hablar» a mano con `nc -C servidor 25` (el `-C` manda CR+LF, que SMTP exige al final de cada línea). Esta es la secuencia, para *leerla* y entender qué hace cada comando:

```
220 mail.ejemplo.com ESMTP Postfix     # saludo del servidor
EHLO mimaquina                         # cliente: me presento (EHLO pide la lista de
extensiones)
250-mail.ejemplo.com                   # servidor: hola, y estas son mis extensiones:
250-SIZE 10240000                      #   tamaño máximo de mensaje
250-STARTTLS                           #   puedo cifrar con TLS
250 SMTPUTF8                           #   acepto UTF-8 en direcciones/headers
MAIL FROM: <remitente>                  # el remitente del SOBRE (Return-Path)
250 2.1.0 Ok
RCPT TO: <destinatario>                 # el destinatario del SOBRE (se puede repetir)
250 2.1.5 Ok
DATA                                   # "ahora viene la carta"
354 End data with <CR><LF>.<CR><LF>    # terminá con una línea que tenga solo un punto
... headers y cuerpo ...
.                                      # un punto solo en una línea = fin del mensaje
250 2.0.0 Ok: queued                   # aceptado para entrega
QUIT
```

🗣 Qué decir

«SMTP separa el sobre, que son MAIL FROM y RCPT TO y se usa para enrutar, de la carta dentro de DATA, que son los headers y el cuerpo que ve el usuario. El servidor va contestando con códigos de tres dígitos; el 354 indica que puedo empezar a escribir el mensaje, que termina con un punto solo en una línea.»

⚠ Relay y por qué el ITBA bloquea SMTP saliente (E55/E56)

Un servidor que acepta reenviar mail de cualquiera hacia cualquier lado es un *open relay* y termina usado para spam. Por eso los servidores solo aceptan entregar a *sus* dominios (desde afuera) o exigen autenticación (submission, 587). El ITBA suele bloquear el puerto 25 saliente para que no se arme un relay desde la red: por eso no podés conectarte a un SMTP externo desde ahí.

MIME: mandar algo más que texto ASCII

SMTP nació para texto ASCII de 7 bits. **MIME** es el estándar que permite mandar acentos, HTML, imágenes y adjuntos describiéndolos con headers:

- `MIME-Version: 1.0`
- `Content-Type`: qué es cada parte. `text/plain; charset="UTF-8"`, `text/html`, `image/png`…
- `Content-Transfer-Encoding`: cómo se codificó para que sobreviva por SMTP: `7bit`, `quoted-printable` o `base64`.
- Tipos **multipart**, que contienen otras partes separadas por un `boundary`:
  - `multipart/alternative`: la misma cosa en varios formatos (texto plano *y* HTML); el MUA muestra el que puede.
  - `multipart/mixed`: cuerpo + adjuntos.
  - `multipart/related`: HTML con imágenes incrustadas (referenciadas por `Content-ID`).
  - `message/rfc822`: un mail completo adjuntado dentro de otro.
- `Content-Disposition: attachment; filename="x.png"`: marcá esta parte como adjunto descargable (vs `inline`). `Content-ID`: identificador para referenciar una imagen desde el HTML (`<img src="cid:...">`).

Estructura típica de un mail con texto, HTML e imagen incrustada:

```
multipart/mixed
├── multipart/related
│   ├── multipart/alternative
│   │   ├── text/plain   (base64)   ← versión texto
│   │   └── text/html    (base64)   ← versión HTML
│   └── image/png        (base64)   ← imagen referenciada por el HTML (cid)
└── (acá irían los adjuntos "sueltos" en un mixed)
```

Codificaciones: base64 y quoted-printable

Ninguna de las dos es cifrado: solo adaptan bytes a caracteres seguros.

- **base64**: toma 3 bytes (24 bits) y los parte en 4 grupos de 6 bits, cada uno mapeado a un carácter del alfabeto `A–Z a–z 0–9 + /`. Crece ~33% (3 bytes → 4 caracteres). El `=` al final es padding. Se usa para binarios (imágenes).
- **quoted-printable**: deja el texto ASCII casi legible y escapa solo los bytes problemáticos como `=XX` (hex). `=C3=A1` es `á` en UTF-8, `=E9` es `é` en ISO-8859-1. Bueno para texto con pocos acentos.

```
printf '¡Hola mundo!' | base64            # wqFIb2xhIG11bmRvIQ==
echo 'wqFIb2xhIG11bmRvIQ==' | base64 -d    # ¡Hola mundo!
# -> para predecir el tamaño: base64 de N bytes ocupa aprox. 4*ceil(N/3) caracteres
```

> 💡 **La cuenta del E65**
>
> Si tu proveedor acepta mails de hasta 10 MB y te mandan una imagen de 9 MB, puede **no entrar**: al codificarla en base64 crece ~33% y queda en ~12 MB. El límite se aplica al mensaje ya codificado.

Decodificar base64 y quoted-printable a mano (E63/E64)

base64 usa este índice: `A`=0 … `Z`=25, `a`=26 … `z`=51, `0`=52 … `9`=61, `+`=62, `/`=63.

```
# Codificar "HO LA" (5 bytes). ASCII: H=72 O=79 ' '=32 L=76 A=65
# bytes:   01001000 01001111 00100000 | 01001100 01000001 (falta 1 byte -> se rellena)
# en 6:    010010 000100 111100 100000 | 010011 000100 0001(00)
# índices:   18     4      60     32   |   19     4     4 -> y como falta, un '='
# base64:    S      E      8      g        T      E      E      =
printf 'HO LA' | base64        # SE8gTEE=
```

POP3: leer el correo

POP3 es otro diálogo de texto (puerto 110). Respuestas `+OK` / `-ERR`. Autenticación con `USER` + `PASS`, y comandos para listar y traer mensajes:

```
nc -C pop3.servidor.com 110
+OK Dovecot ready
USER mi-usuario          # mi casilla
PASS mi-clave            # (en POP3 plano viaja en claro: por eso se usa POP3S/995 o STARTTLS)
STAT                     # cuántos mensajes y cuántos bytes
LIST                     # lista: número y tamaño de cada mensaje
RETR 1                   # traeme el mensaje 1 (headers + cuerpo, tal cual, en MIME)
DELE 1                   # marcarlo para borrar
QUIT                     # cierra y aplica los DELE
```

Después de un `RETR`, si el cuerpo es `text/plain; base64`, copiás esa parte y la pasás por `base64 -d` para leerla; si tiene una imagen, guardás la parte `image/...` y la decodificás a un archivo.

Antispam: SPF, DKIM, DMARC (E61/E62)

Técnica	Qué verifica	Cómo
SPF	Que la IP que entrega el mail esté autorizada a enviar por ese dominio	Registro TXT con `v=spf1 ...`. El servidor receptor compara la IP del emisor contra la lista
DKIM	Que el mensaje no se modificó y proviene del dominio	El emisor **firma** partes del mail con su clave privada; la pública está en un TXT. El receptor verifica la firma
DMARC	Qué hacer si falla SPF/DKIM, y que el dominio del `From:` coincida	Registro TXT con política: `none`, `quarantine` o `reject`, y a dónde reportar
Greylisting	Frenar spammers que no reintentan	Rechazar temporalmente (4xx) la primera vez; un MTA legítimo reintenta y pasa
Filtros bayesianos	Clasificar por contenido	Probabilidad de spam aprendida de palabras y patrones

Una política SPF se lee así:

```
dig TXT proto.leak.com.ar +short
#   "v=spf1 ip4:192.168.122.235 -all"
#    v=spf1           -> es SPF versión 1
#    ip4:192.168...   -> mecanismo: esa IP está autorizada a enviar (puede haber varios, o
include:)
#    -all             -> cualquier otra IP: FAIL (rechazar). ~all = softfail, ?all = neutral
dig TXT itba.edu.ar +short      # E62: las IPs/includes listados son las que pueden enviar por
itba.edu.ar
```

Ejercicio de parcial resuelto

Recuperatorio — Ejercicio I

Sobre `proto.leak.com.ar`: A. ¿Cuáles son las IPs de los servidores SMTP que reciben el correo entrante? B. Según la política SPF, ¿qué IPs tienen permitido enviar correos salientes?

```
dig MX proto.leak.com.ar +short         # 20 smtp20.proto.leak.com.ar. / 30
smtp30.proto.leak.com.ar.
dig +short smtp20.proto.leak.com.ar     # 127.0.0.1
dig +short smtp30.proto.leak.com.ar     # 192.168.122.235
dig TXT proto.leak.com.ar +short        # "v=spf1 ip4:192.168.122.235 -all"
# A (entrante, de los MX): 127.0.0.1 y 192.168.122.235
# B (saliente, del SPF):   solo 192.168.122.235
```

🗣 Qué decir

«El correo entrante lo definen los MX, que resuelvo a IPs con consultas A. El saliente lo define SPF, publicado en un TXT: el mecanismo ip4 enumera las IPs autorizadas y -all indica rechazar cualquier otra. Son listas distintas: entrante y saliente no tienen por qué coincidir.»

✍ Practicá en papel — Capítulo 8

1. Si el `RCPT TO` dice `a@x.com` y el `To:` dentro de DATA dice `b@y.com`, ¿a quién llega el mail? ¿Qué remitente ve el usuario si el `From:` difiere del `MAIL FROM`?
2. Codificá `chau` en base64 a mano (c=99, h=104, a=97, u=117). Verificá el tamaño esperado.
3. ¿Qué significa `=C3=A9` en un cuerpo quoted-printable UTF-8?
4. Un mail tiene `multipart/alternative` con `text/plain` y `text/html`. ¿Por qué se incluyen los dos?
5. Un dominio tiene `v=spf1 include:_spf.google.com ~all`. ¿Qué pasa con un mail enviado desde una IP no listada?
6. ¿Por qué el ITBA bloquea el SMTP saliente a servidores externos?

AMPLIACIÓN DEL CAPÍTULO 8 · CORREO

Variantes de SMTP/POP3 y un ejercicio resuelto

El diálogo SMTP/POP3 es siempre el mismo; lo que cambian en el parcial son los requisitos del mensaje. Acá está qué te pueden pedir y qué agregás, cómo leer el buzón con `curl`, y un ejercicio de la cátedra resuelto de punta a punta.

Qué te pueden cambiar (y qué agregás al mail)

SI TE PIDEN…	AGREGÁS EN LOS HEADERS DE DATA / EL CUERPO
texto con acentos / ñ / símbolos	`Content-Type: text/plain; charset=UTF-8` + `Content-Transfer-Encoding: base64` (o `quoted-printable`), y codificás el cuerpo.
el cuerpo en **negrita**, subrayado, con formato	`Content-Type: text/html; charset=UTF-8` y en el cuerpo `<b>…</b>`, `<u>…</u>`.
texto plano **y** HTML (que el MUA elija)	`multipart/alternative` con dos partes: `text/plain` y `text/html`, separadas por un `boundary`.
adjuntar un archivo / una imagen suelta	`multipart/mixed`; la parte del adjunto con `Content-Disposition: attachment; filename="x.png"` y `base64`.
una imagen **incrustada** en el HTML	`multipart/related`; la imagen con `Content-ID: <img1>` y en el HTML `<img src="cid:img1">`.
mandar a varios destinatarios	repetís `RCPT TO:<…>` por cada uno (sobre); los `To:` / `Cc:` de DATA son aparte.
que el servidor exija autenticación	submission en **587**: `AUTH LOGIN` y usuario/clave en base64 (o `curl --user`).

⚠ Dos cosas de SMTP a mano

Cada línea termina en CR+LF: por eso `nc -C`. Y una línea que empieza con un punto dentro de DATA hay que escribirla con **dos** puntos (*dot-stuffing*); si el cuerpo va en base64 esto no aparece. El mensaje termina con un punto solo en su línea.

Leer el buzón con curl (además de nc)

POP3 también se habla con `curl`, que arma el `USER/PASS/LIST/RETR` por vos:

```
curl -v --user "5" pop3://192.168.0.238:110/        # sin nro de mensaje = LIST (lista nro y tamaño)
curl -v --user "5" pop3://192.168.0.238:110/1       # /1 = RETR 1: trae el mensaje 1 completo
# la contraseña la pide aparte (no se ve al tipearla). El formato de LIST es:  numero tamaño
```

La autenticación PLAIN de POP3 usa base64, que **no cifra**: la clave viaja legible (por eso existen POP3S/995 y STARTTLS).

Ejercicio resuelto · POP3 + base64 + SMTP

De la cátedra: entrar al buzón por POP3, leer el mensaje 1, decodificar una clave que viene en base64, **responder** por SMTP devolviendo esa clave con la codificación correcta, y borrar el original. Servidor `192.168.0.238`, usuario `5`, responder a `secretary@usa.com`.

1 · Listar y leer el mensaje

```
curl -v --user "5" pop3://192.168.0.238:110/       # → LIST:  1 955   (un mensaje, 955 bytes)
curl -v --user "5" pop3://192.168.0.238:110/1      # RETR 1: headers (From/To/Subject) + cuerpo
```

El número 1 no se adivina: sale del `LIST`, cuyo formato es `numero tamaño`.

2 · Decodificar la clave (viene en base64)

```
base64 -d <<'EOF'
TG9zIG1pc2lsZXMg...  (el bloque base64 del cuerpo)
EOF
# El texto decodificado trae la consigna y la clave a devolver. La respuesta del acertijo es
# exactamente la cadena:   .ñáéó¡¿   (caracteres fuera de ASCII: por eso importa la codificación)
```

3 · Preparar la clave para mandarla

```
printf '%s' '.ñáéó¡¿' | base64        # → LsOxw6HDqcOzwqHCvw==   (usamos printf, no echo: echo agrega un \n)
```

La clave tiene caracteres fuera de ASCII. MIME deja declarar `charset=UTF-8` y transporte `base64`, y así el cuerpo viaja como ASCII seguro. **Solo** se codifica el cuerpo; los headers MIME van legibles.

4 · Conectarse y enviar por SMTP

```
nc -C 192.168.0.238 25
220 … ESMTP   (esperá el saludo)
EHLO localhost                        # → 250 … (capacidades)
MAIL FROM:<5@imf.com>                  # → 250  (remitente del SOBRE)
RCPT TO:<secretary@usa.com>            # → 250  (destinatario del SOBRE)
DATA                                  # → 354  "ahora viene la carta"
# a partir de acá, headers de la carta + línea vacía + cuerpo:
From: 5@imf.com
To: secretary@usa.com
Subject: Re: Mission: Impossible
MIME-Version: 1.0
Content-Type: text/plain; charset=UTF-8
Content-Transfer-Encoding: base64
(línea vacía obligatoria: separa headers y cuerpo)
LsOxw6HDqcOzwqHCvw==
.                                     # punto solo = fin del mensaje  → 250 queued
QUIT                                  # → 221 cierre
```

> **💡 El 250 que importa es el del final**
>
> Hay varios 250: tras `MAIL FROM` (aceptó el remitente), tras `RCPT TO` (el destinatario) y el **250 después del punto** (aceptó el mensaje completo). Entre medio, el `354` habilita escribir la carta. El 250 final confirma aceptación por SMTP; no prueba que lo hayan leído.

5 · Borrar el original (lo pide la consigna)

```
nc -C 192.168.0.238 110
USER 5            # → +OK
PASS tu-clave     # → +OK   (acá la clave SÍ se ve; es la del buzón, no la del acertijo)
LIST              # confirmá cuál es la misión
RETR 1            # verificá el contenido antes de borrar
DELE 1            # marca para borrar
QUIT              # → en POP3 el borrado se aplica recién al QUIT (DELE solo marca; RSET lo deshace)
```

> **🗣 Qué decir**
>
> «Leí el buzón por POP3 con LIST y RETR, decodifiqué el cuerpo en base64. Para responder usé SMTP separando sobre (MAIL FROM/RCPT TO) y carta (DATA): como la clave tiene caracteres no ASCII, declaré `charset=UTF-8` y `Content-Transfer-Encoding: base64` y codifiqué solo el cuerpo. El 250 posterior al punto confirmó la aceptación, y borré el original con DELE + QUIT.»

9 DHCP: configuración automática de red

La idea

Un host recién conectado no tiene IP, no conoce su máscara, ni su gateway, ni su DNS. DHCP (sobre UDP, servidor en el puerto **67**, cliente en el **68**) se los entrega «prestados» por un tiempo: el **lease**.

DORA: los cuatro mensajes

#	Mensaje	De → a	Qué dice
1	**D**iscover	0.0.0.0:68 → 255.255.255.255:67	«¿Hay algún servidor DHCP? Esta es mi MAC.» Puede pedir una IP preferida (option 50)
2	**O**ffer	servidor → broadcast (o unicast a la MAC)	«Te ofrezco esta IP (`yiaddr`), con esta máscara, gateway, DNS y lease»
3	**R**equest	0.0.0.0 → 255.255.255.255	«Acepto la oferta del servidor X» (option 54 Server Identifier). Va en broadcast para que los demás servidores sepan que no los eligió
4	**A**ck	servidor → cliente	«Confirmado, es tuya por N segundos». Recién ahí el cliente configura la interfaz

💡 **Preguntas que siempre hacen**

- **¿Por qué broadcast?** El cliente no tiene IP ni sabe la del servidor. Manda desde `0.0.0.0` a `255.255.255.255`, y la trama Ethernet va a `ff:ff:ff:ff:ff:ff`. El servidor responde en broadcast, o en unicast directo a la MAC del cliente (campo `chaddr`) si este no puso el flag de broadcast.
- **Transaction ID (`xid`)**: número aleatorio que elige el cliente y se mantiene en los 4 mensajes. Como todo va en broadcast, sirve para emparejar respuestas con pedidos.
- **Renovación**: al 50% del lease (T1) el cliente pide renovar en unicast a su servidor (Request → Ack). Si no le contestan, al 87,5% (T2) prueba en broadcast con cualquier servidor (rebinding). Si se vence, suelta la IP y vuelve a empezar con Discover.
- **Release**: el cliente avisa que deja la IP (`dhclient -r`). **NAK**: el servidor rechaza un Request (por ejemplo, IP de otra red).
- **Relay**: los broadcasts no cruzan routers; si el servidor está en otra red, un relay agent reenvía los mensajes.

Opciones más comunes

Nº	Opción	Nº	Opción
1	Máscara de subred	51	Lease time
3	Router (gateway)	53	Tipo de mensaje (1 Discover, 2 Offer, 3 Request, 5 Ack, 6 Nak, 7 Release)
6	Servidores DNS	54	Server Identifier (IP del servidor)
15	Nombre de dominio	50 · 55	IP pedida · lista de parámetros que pide el cliente

isc-dhcp-server

```
sudo apt install isc-dhcp-server
/etc/dhcp/dhcpd.conf              # la configuración
/etc/default/isc-dhcp-server      # en qué interfaz escuchar: INTERFACESv4="enp0s8"
/var/lib/dhcp/dhcpd.leases        # las IPs entregadas (para ver qué asignó)
sudo dhcpd -t -cf /etc/dhcp/dhcpd.conf   # valida la sintaxis
sudo systemctl restart isc-dhcp-server
journalctl -u isc-dhcp-server -e         # errores
```

⚠ El error más común

La interfaz donde escucha el servidor tiene que tener una **IP fija dentro de la subred** que declarás. Si no, el servicio no arranca y el log dice *«No subnet declaration for enp0s8»*. Y nunca lo hagas sobre la red del laboratorio o de tu casa: le estarías repartiendo IPs a otros. Usá una red interna o host-only de la VM.

Ejercicio de parcial resuelto

Parcial 2Q2025 — Ejercicio I

Red `192.168.100.0/24` con servidor DHCP. 1) Una impresora con MAC `08:00:27:AE:C8:6A` debe tener siempre la IP `192.168.100.8`. 2) Todos los clientes usan los DNS de Google (`8.8.8.8` y `8.8.4.4`). 3) Rango de asignación 10–200; el resto queda reservado. Verificar con `sudo nmap --script broadcast-dhcp-discover --script-args "broadcast-dhcp-discover.mac=AA:BB:CC:DD:EE:FF"`. No hacerlo sobre la Wi-Fi del laboratorio: usar una red interna o interfaz virtual.

```
# 0) Interfaz de la red interna con IP fija dentro de la subred (el servidor es .1)
sudo ip addr add 192.168.100.1/24 dev enp0s8
sudo ip link set enp0s8 up

# 1) /etc/default/isc-dhcp-server
INTERFACESv4="enp0s8"             # solo atiende esta interfaz: no molesta a otras redes
```

```
# 2) /etc/dhcp/dhcpd.conf
authoritative;                    # soy EL servidor de esta red: puedo mandar NAK a pedidos
inválidos
ddns-update-style none;           # no intentar actualizar DNS dinámicamente
default-lease-time 600;           # lease de 10 minutos si el cliente no pide otro
max-lease-time 7200;

option domain-name-servers 8.8.8.8, 8.8.4.4;   # requisito 2, para TODOS los clientes (global)

subnet 192.168.100.0 netmask 255.255.255.0 {
    range 192.168.100.10 192.168.100.200;      # requisito 3: solo reparte de .10 a .200
    option routers 192.168.100.1;              # gateway
    option subnet-mask 255.255.255.0;
    option broadcast-address 192.168.100.255;
}

host impresora {                               # requisito 1: reserva por MAC
    hardware ethernet 08:00:27:AE:C8:6A;
    fixed-address 192.168.100.8;               # FUERA del range: así nunca se la dan a otro
}
```

```
# 3) Validar, reiniciar y verificar
sudo dhcpd -t -cf /etc/dhcp/dhcpd.conf && sudo systemctl restart isc-dhcp-server
sudo nmap -e enp0s8 --script broadcast-dhcp-discover \
     --script-args broadcast-dhcp-discover.mac=AA:BB:CC:DD:EE:FF
#   IP Offered: 192.168.100.10   Domain Name Server: 8.8.8.8, 8.8.4.4   Router: 192.168.100.1
sudo nmap -e enp0s8 --script broadcast-dhcp-discover \
     --script-args broadcast-dhcp-discover.mac=08:00:27:AE:C8:6A
#   IP Offered: 192.168.100.8    <- la reserva funciona
# -e enp0s8: por qué interfaz mandar el Discover. El script simula un cliente: manda un
DHCPDISCOVER
#            con la MAC indicada y muestra lo que el servidor ofrece
```

🗣 Qué decir

«El servidor solo escucha en la interfaz de la red interna, que tiene IP fija dentro de la subred. Las opciones globales, como los DNS de Google, se envían a todos los clientes; el range limita la asignación dinámica de .10 a .200, y la impresora tiene una reserva por MAC con un fixed-address fuera del rango, para que esa IP nunca se le entregue a otro. Lo verifico con nmap, que simula un DHCP Discover con una MAC cualquiera y con la de la impresora y muestra el Offer.»

✍ Practicá en papel — Capítulo 9

1. Escribí de memoria el `dhcpd.conf` para la red `10.1.1.0/24`, rango .50–.99, gateway .1, lease de 2 minutos.
2. ¿Qué IP origen y destino tiene un DHCP Request? ¿Por qué?
3. Con lease de 120 s, ¿en qué segundo intenta renovar el cliente? ¿Y el rebinding?
4. ¿Por qué conviene que la `fixed-address` quede fuera del `range`?
5. ¿Cómo le negarías la configuración a una MAC específica? (pista: `deny booting` / `ignore booting` dentro de un `host`).

10 SSH: login remoto y túneles

La idea

SSH da una terminal cifrada en una máquina remota (reemplaza a Telnet, que va todo en texto plano). Autentica al **servidor** con su clave de host (de ahí el `known_hosts`) y al **usuario** por contraseña o por **clave pública**. Además sabe **tunelizar**: meter otras conexiones TCP dentro del canal cifrado, que es lo que más toman.

Autenticación por clave

Generás un par (`ssh-keygen`): la **privada** queda en tu máquina (`~/.ssh/id_rsa`), la **pública** la copiás al servidor (`~/.ssh/authorized_keys`). Al conectarte, el servidor te desafía a probar que tenés la privada, sin que esta salga nunca. La privada puede estar protegida por una **passphrase** (una clave que la descifra localmente): eso es justo lo que hay que descubrir en los ejercicios de la llave.

```
ssh usuario@host                          # login (puerto 22 por defecto)
ssh -p 2222 usuario@host                  # -p: otro puerto
ssh -i llave.pem usuario@host             # -i: usar esta clave privada
scp archivo usuario@host:/ruta/           # copiar archivos por SSH (E101: traer /etc/passwd de pampero)
ssh-keygen -l -f /etc/ssh/ssh_host_rsa_key.pub   # fingerprint de la clave del servidor
cat ~/.ssh/known_hosts                    # claves de servidores ya aceptados
```

⚠ Permisos de la llave privada

SSH se niega a usar una clave que puedan leer otros: *«UNPROTECTED PRIVATE KEY FILE»*. Hay que dejarla solo para el dueño:

```
chmod 600 llave.pem      # dueño rw, nadie más. (equivalente: chmod og-rwx llave.pem)
```

💡 known_hosts (E99)

La primera vez SSH no conoce al servidor y te muestra su fingerprint para que lo aceptes; lo guarda en `known_hosts`. Si después la clave del host cambia (o un nombre pasa a apuntar a otra máquina), SSH grita *«REMOTE HOST IDENTIFICATION HAS CHANGED»*: protege contra un impostor en el medio.

Túneles: los tres tipos

La clave para no marearse: **el túnel se crea desde el cliente SSH**, y lo que define el tipo es *quién abre el puerto de entrada* y *hacia dónde sale*.

-L (local): «traer» un servicio remoto a mi máquina

Abre un puerto **en mi máquina**. Lo que entra ahí sale **por el servidor SSH** hacia un destino.

```
ssh -L 12345:localhost:12345 usuario@pampero.itba.edu.ar
#      └──┬──┘ └─────┬──────┘
#   mi puerto    destino visto DESDE pampero (localhost = la propia pampero)
# Resultado: conectarme a MI 127.0.0.1:12345 es como conectarme al 12345 de pampero.
# "quiero llegar a un servicio del servidor como si fuera local"
```

-R (remoto): «publicar» un servicio mío en el servidor

Abre un puerto **en el servidor remoto**. Lo que entra ahí sale **por mi máquina** hacia un destino.

```
ssh -R 9090:localhost:9090 usuario@pampero.itba.edu.ar
#      └──┬──┘ └─────┬──────┘
# puerto en pampero   destino visto DESDE mi máquina
# Resultado: alguien en pampero que se conecte a su 127.0.0.1:9090 llega a MI 9090.
# "exponer un servicio de mi máquina en el servidor" (E103)
```

-D (dinámico): montar un proxy SOCKS

Abre un puerto SOCKS en mi máquina; cada conexión que le mando sale **por el servidor SSH** hacia donde yo pida. Es un proxy genérico.

```
ssh -D 9090 usuario@pampero.itba.edu.ar          # proxy SOCKS5 en mi 127.0.0.1:9090
curl -x socks5h://127.0.0.1:9090 https://api.ipify.org/   # -> devuelve la IP de pampero, no la mía
# "navegar con la IP del servidor" (E102): todo el tráfico sale desde pampero
```

Flags útiles: `-N` (no abrir shell, solo el túnel), `-f` (a segundo plano), `-g` (permitir que otros usen el puerto local).

> 💡 **Truco para no confundir -L y -R**
>
> La letra indica dónde se abre el puerto de entrada respecto del *comando*: **L**ocal = en tu máquina (la que ejecuta ssh), **R**emoto = en el servidor. La sintaxis siempre es `puerto_de_entrada:destino_host:destino_puerto`, y el `destino` se interpreta desde el *otro* extremo del túnel.

> ⚠ **Por qué falla `ssh localhost -L1234:127.0.0.1:1234` (ejercicio de repaso)**
>
> Abre el puerto 1234 local y lo manda… al 1234 de localhost, que es el mismo puerto que acabás de abrir: el túnel apunta a sí mismo. No hay ningún servicio real detrás, así que una conexión entra en un lazo o no llega a nada.

Ejercicios de parciales resueltos

Parciales 2021/recus — Ejercicio de la llave con passphrase

Obtenga control de la línea de comandos del usuario `pdc` del host `parcial.leak.com.ar` (servidor SSH en el puerto 2222). Una llave privada está en `http://parcial.leak.com.ar/llave.pem`, protegida por passphrase. La passphrase es la dirección MAC (17 caracteres `xx:xx:xx:xx:xx:xx`) del gateway del host `pampero.itba.edu.ar`.

Dos subproblemas: (1) descubrir la passphrase, que es una MAC, y (2) usar la llave para entrar.

```
# (1) La passphrase = MAC del gateway de pampero  -> capítulo 2
ssh usuario@pampero.itba.edu.ar
route -n                        # la línea default (0.0.0.0) da la IP del gateway, ej
10.16.1.254
ping -c1 10.16.1.254            # fuerzo el ARP por si no está en la tabla
arp -n | grep 10.16.1.254       # la MAC, ej 00:87:31:e1:0f:d2  <- esta es la passphrase

# (2) Descargar la llave, arreglar permisos y entrar
wget http://parcial.leak.com.ar/llave.pem
chmod 600 llave.pem
ssh -i llave.pem -p 2222 pdc@parcial.leak.com.ar
#   Enter passphrase for key 'llave.pem':  -> escribo la MAC
date                            # ya adentro: demuestro control ejecutando un comando
```

En la variante del DHCP (Parcial 1C2021), la passphrase es la IP (11 caracteres) que DHCP le asignó al cliente en un pcap: se saca del Ack del pcap (capítulo 10). En otra variante la MAC es del host `bd1` accesible desde pampero: se hace `ping bd1` y `arp -n | grep <ip>`.

🗣 Qué decir

«Obtener la shell se hace con ssh, usando -i para la llave privada y -p para el puerto 2222. La llave está protegida por passphrase, que resultó ser la MAC del gateway de pampero: me conecté a pampero, miré la ruta default para la IP del gateway, forcé el ARP con un ping y leí la MAC en la tabla ARP. Descargué la llave con wget, le puse permisos 600 porque SSH rechaza llaves legibles por otros, y entré.»

Parciales — `tail -f` remoto con túneles

Realice los cambios necesarios para que al ejecutar `tail -f archivo.log` en una terminal de su host pueda seguir en vivo los cambios del archivo `/tmp/pdc` (o `/tmp/bar`) del host `pampero.itba.edu.ar`. Cualquier solución que no use túneles obtiene la mitad del puntaje. Use su número de legajo como puerto para evitar colisiones.

Idea: en pampero, un `tail -f` va empujando las líneas nuevas por un netcat; esos bytes viajan por el túnel SSH hasta tu máquina, donde otro netcat los vuelca en `archivo.log`, que vos seguís con `tail -f`. Con un **túnel local (-L)**:

```
# Terminal 1: abro el túnel. Mi puerto LEGAJO sale hacia el LEGAJO de pampero
ssh -L LEGAJO:localhost:LEGAJO usuario@pampero.itba.edu.ar
#   (quedo con una shell en pampero)  En esa shell: empujo el archivo por el puerto
tail -f /tmp/pdc | nc -l localhost LEGAJO        # nc escucha en pampero y manda lo que le
entra por el tail

# Terminal 2 (mi máquina): leo del extremo local del túnel y guardo
nc localhost LEGAJO > archivo.log

# Terminal 3 (mi máquina): sigo el archivo
tail -f archivo.log
```

El reglamento sugiere generar datos de prueba con `while true; do date; sleep 1; done > /tmp/pdc`.

🗣 Qué decir

«Armé un túnel SSH local: mi puerto LEGAJO queda conectado, a través del canal cifrado, al mismo puerto de pampero. En pampero, tail -f empuja las líneas nuevas a un netcat que escucha en ese puerto; los bytes cruzan el túnel y en mi máquina otro netcat los escribe en archivo.log, que sigo con tail -f. Uso túnel porque sin él sería la mitad del puntaje.»

Parcial 2Q2025 Ej. IV / recus — dar Internet a un host sin NAT

El host `parcial.leak.com.ar` tiene una única interfaz con IP privada y su gateway no hace NAT. Realice los cambios para que desde su línea de comandos se pueda acceder con curl a cualquier sitio de Internet (por ejemplo `http://api.ipify.org` y `https://campus.itba.edu.ar/`). Sin túneles, la mitad del puntaje. (Requiere tener acceso por SSH al host, del ejercicio anterior.)

El host no sale a Internet, pero vos (desde donde estás, que sí tenés salida) sí. La idea: montar un proxy SOCKS que salga por tu máquina y hacer que el host lo use. Se combina **-R** (publicar una entrada en el host remoto) con **-D** (proxy que sale por vos).

```
# En tu máquina necesitás un servidor SSH local corriendo
sudo systemctl start ssh

# 1) Publicás en 'parcial' un puerto (9999) que sale por tu máquina, y ahí montás un proxy dinámico
#    truco: el túnel reverso apunta a un -D de tu propia máquina
ssh -R 9999:localhost:9090 pdc@parcial.leak.com.ar    # (terminal con shell en parcial) el 9999 de parcial -> tu 9090
# 2) En tu máquina, abrís el proxy SOCKS en 9090
ssh -D 9090 localhost
# 3) Desde la shell en 'parcial', salís usando ese proxy
ALL_PROXY=socks5h://127.0.0.1:9999 curl http://api.ipify.org/     # -> devuelve TU IP pública
ALL_PROXY=socks5h://127.0.0.1:9999 curl https://campus.itba.edu.ar/
```

💡 Por qué socks5h y ALL_PROXY

`ALL_PROXY` es la variable de entorno que curl lee para usar un proxy sin pasar `-x`. `socks5h` hace que el **nombre** (campus.itba.edu.ar) lo resuelva el otro extremo del proxy, que es el que tiene DNS y salida. Sin la `h`, el host intentaría resolver el nombre él mismo y, como no tiene Internet, fallaría.

🗣 Qué decir

«El host no tiene salida a Internet, pero yo sí. Monté un proxy SOCKS dinámico con ssh -D que sale por mi máquina, y con un túnel reverso ssh -R publiqué en el host un puerto que desemboca en ese proxy. Entonces desde el host hago curl con ALL_PROXY apuntando a ese puerto: el tráfico viaja por el túnel y sale a Internet desde mi conexión. Uso socks5h para que la resolución de nombres también ocurra de mi lado. La prueba es que api.ipify.org devuelve mi IP pública y no la del host.»

✍ Practicá en papel — Capítulo 10

1. ¿Qué hace `ssh -L 8080:localhost:80 user@web`? ¿A qué te conectás en tu máquina y qué ves?
2. ¿Y `ssh -R 8080:localhost:3000 user@server`? ¿Quién puede usar ese 8080 y a dónde llega?
3. Querés navegar como si estuvieras en la red del ITBA. ¿Qué comando y qué curl?
4. ¿Por qué `chmod 600` en la llave? ¿Qué error da si no?
5. Explicá por qué `ssh localhost -L1234:127.0.0.1:1234` no sirve para nada.
6. Pampero tiene un proxy SOCKS en 1080. ¿Por qué `curl host.interno -x socks5://pampero:1080` anda en el ITBA y falla desde casa? ¿Cambiás algo con `socks5h`?

11 Wireshark y análisis de capturas

La idea

Un **pcap** es una grabación de los paquetes que pasaron por una interfaz. Wireshark los muestra y permite filtrarlos, seguir conversaciones y extraer lo que viajó (texto, imágenes, credenciales). Los ejercicios de pcap siempre son: *filtrar* para encontrar los paquetes relevantes, *seguir el stream* para leer la conversación completa, y a veces *decodificar* lo que se encontró.

Capturar (si tuvieras que hacerlo)

```
sudo tcpdump -i enp0s3 -w captura.pcap          # grabar en archivo
sudo tcpdump -i enp0s3 port 53 -w dns.pcap      # solo DNS
zcat captura.pcap.gz > captura.pcap             # los pcap del parcial vienen comprimidos .gz
wireshark captura.pcap                          # abrir en la interfaz gráfica
```

Filtros de display (los que se escriben en la barra de Wireshark)

Filtro	Qué muestra
`dhcp` (o `bootp`)	Los 4 mensajes DORA
`dns`	Consultas y respuestas DNS
`http` · `http.request`	HTTP · solo los requests
`smtp` · `pop` · `imap`	Correo
`tcp.port == 25` · `udp.port == 520`	Por puerto (SMTP · RIP)
`ip.addr == 192.168.0.1`	Todo lo que involucra esa IP (origen o destino)
`ip.src == X && ip.dst == Y`	De X hacia Y
`icmp`	ping, Time Exceeded (traceroute), Port/Host Unreachable
`arp`	Who-has / is-at
`tcp.stream == 3`	Una conversación TCP concreta
`frame contains "password"`	Paquetes que contienen ese texto

Follow → TCP Stream (clic derecho sobre un paquete) reconstruye toda la conversación de esa conexión en una ventana: lo que mandó el cliente y lo que respondió el servidor. Es lo que se usa para leer un diálogo SMTP o HTTP completo. Se puede guardar como texto (Save as → raw) y procesarlo.

Equivalentes por línea de comandos (tshark)

```
tshark -r captura.pcap -Y 'dhcp'                          # -Y = filtro de display
tshark -r captura.pcap -Y 'dns' -T fields -e dns.qry.name -e dns.a   # extraer campos
tshark -r captura.pcap -z follow,tcp,ascii,3              # seguir el stream TCP número 3
tshark -r captura.pcap -q -z conv,tcp                     # resumen de conversaciones
```

Las tres capturas del parcial, resueltas

Estos son los pcaps que circulan (`dhcp-parcial.pcap`, `parcial-correo.pcap`, `botica.pcap`). Resueltos con los valores reales, para que sepas qué buscar y qué te va a dar.

dhcp-parcial.pcap — la IP que asignó DHCP

> **Uso típico**
> La passphrase de la llave SSH es la IP (11 caracteres) que DHCP le asignó al cliente en esta secuencia.

Filtro `dhcp`: aparecen exactamente los 4 mensajes DORA. La IP asignada está en el campo **`Your (client) IP address`** (`yiaddr`) del **Ack** (y del Offer).

```
Discover  de 00:50:ba:12:47:cb (host "academy04"), xid 0xfe089c15, pide 10.20.20.20
Offer     de 10.20.20.4  -> yiaddr = 10.20.20.20   (DNS 10.20.20.1, máscara /24, lease 300s,
dominio far-far-away)
Request   el cliente confirma 10.20.20.20 al server 10.20.20.4
Ack       de 10.20.20.4  -> yiaddr = 10.20.20.20   ✔ IP asignada = 10.20.20.20
```

En Wireshark: expandir el Ack → *Bootstrap Protocol* → *Your (client) IP address: 10.20.20.20*. Esa es la respuesta (en este pcap son 11 caracteres solo si la IP los tiene; el método es el mismo).

> **🗣 Qué decir**
> «Filtré por dhcp y encontré el intercambio DORA. La IP que el servidor asigna viaja en el campo yiaddr; la leo del Ack, que es la confirmación final, no del Discover. Acá fue 10.20.20.20, con el servidor DHCP en 10.20.20.4.»

parcial-correo.pcap — el correo con una imagen

> **Enunciado**
> Contiene un correo transportado por alguno de los protocolos vistos. El correo contiene una imagen. Mostrarla.

Filtro `smtp` (el correo viaja por SMTP, puerto 25). **Follow → TCP Stream** sobre cualquier paquete del DATA reconstruye el mail completo. Dentro hay una estructura MIME `multipart` con una parte `image/png` codificada en base64.

```
Stream SMTP (192.168.122.1 -> 192.168.122.235:25):
  EHLO ... / MAIL FROM / RCPT TO / DATA
  Subject: feliz parcial
  Content-Type: multipart/mixed; boundary="..."
    ... multipart/alternative: text/plain "Espero que te encuentres entretenido." + emojis
    ... image/png; name="image.png"; Content-Transfer-Encoding: base64
        iVBORw0KGgoAAAANSUhEUgAA...   (los bytes de la imagen)
```

Pasos para sacar la imagen:

```
# En Wireshark: File -> Export Objects -> IMF/HTTP, o Follow TCP Stream -> Save
# A mano, una vez guardada la parte base64 de la imagen en image.txt (solo las líneas base64):
cat image.txt | base64 -d > image.png
xdg-open image.png      # (o eog image.png) -> se abre la imagen
```

En este pcap la imagen es un flyer del **«Observatorio de Logística Sustentable (OLS) – Primer encuentro bimensual 2020, Economía Circular, 5/MAY 09:30–11:30»** del ITBA.

> **⚠ Detalle**
>
> Copiá **solo** las líneas en base64 de esa parte (entre su header y el próximo `boundary`), sin los headers MIME ni el boundary, o `base64 -d` falla. Si el PNG sale «roto», suele ser que incluiste una línea de más o faltó una.

> **🗣 Qué decir**
>
> «El correo viaja por SMTP, así que filtré por smtp y seguí el stream TCP para ver el mensaje completo. Es un MIME multipart con una parte image/png codificada en base64. Extraje esa parte y la decodifiqué con base64 -d a un archivo PNG, que al abrirlo muestra la imagen.»

botica.pcap — captura mixta (DNS, Telnet, SMTP, RIP, traceroute)

> **Enunciado (variante)**
>
> Contiene pedidos/respuestas DNS y tramas de otros protocolos. Responder, por ejemplo: a qué redes tiene acceso `10.0.0.1`; qué DNS propone un servidor; a cuántos saltos está 8.8.8.8; la password de un login remoto; el cuerpo de un correo.

Esta captura es un «cajón de sastre». Cómo atacar cada pregunta:

Pregunta	Filtro / técnica	Resultado en este pcap
¿A qué redes tiene acceso un router?	`rip` (udp 520): las rutas que *anuncia* son las redes que alcanza	`10.0.0.1` anuncia `200.0.1.0` y `10.0.0.2` anuncia `200.0.2.0` (métrica 1)
¿Qué DNS propone el servidor DHCP?	`dhcp` → Offer/Ack → option *Domain Name Server*	`10.20.20.1` (servidor DHCP `10.20.20.4`)
¿A cuántos saltos está 8.8.8.8?	`icmp`: contar los Time Exceeded del traceroute hasta el Port/Dest Unreachable final de 8.8.8.8	Se cuentan los saltos distintos hasta que 8.8.8.8 responde (Destination Unreachable)
Password de login remoto	`telnet` + Follow TCP Stream (Telnet va en texto plano)	login `fake`, el stream muestra el `Password:` tecleado (lo lee cualquiera: por eso Telnet es inseguro)
Cuerpo del correo	`smtp` → Follow Stream → base64 -d de la parte de texto	texto en base64 en el cuerpo del mail

💡 Por qué RIP responde «a qué redes llega»

RIP es un protocolo de ruteo: cada router anuncia por broadcast (UDP 520) las redes que conoce y con qué métrica (saltos). Mirando los *RIP Response* que emite un router ves exactamente su tabla, es decir a qué redes tiene acceso. Una métrica **16** significa «inalcanzable» (infinito en RIP).

🗣 Qué decir (Telnet)

«Telnet no cifra nada: filtré por telnet, seguí el stream TCP y la conversación muestra el usuario y la contraseña tal como se tipearon. Esto ilustra por qué Telnet se reemplazó por SSH, que cifra el canal.»

✍ Practicá en papel — Capítulo 11

1. Escribí el filtro para ver solo los requests HTTP hacia `10.0.0.5`.
2. Querés leer un diálogo SMTP completo en un pcap. ¿Qué dos acciones hacés en Wireshark?
3. En un DHCP, ¿en qué campo y en qué mensaje está la IP asignada al cliente?
4. Ves 3 paquetes ICMP Time Exceeded de 3 IPs distintas y después un Destination Unreachable. ¿Qué herramienta generó eso y cuántos saltos hay al destino?
5. ¿Por qué en un pcap podés leer una password de Telnet pero no una de SSH?
6. Tenés la parte base64 de una imagen en un mail. Escribí el comando para convertirla en PNG.

12 Respuestas de la práctica

Tratá de resolverlas antes de mirar. Si algo no te cierra, volvé al capítulo correspondiente: la idea es entender el porqué, no la respuesta.

Capítulo 1 — Kit de Linux

1. `-rw-r-----` = **640** (dueño rw=6, grupo r=4, otros nada=0). Solo vos lee/escribe = **600**.
2. `ping foo` funciona (usa `/etc/hosts`, que resuelve `foo→127.0.0.1`). `dig foo` **no** lo resuelve o da otra cosa: dig le pregunta directo al servidor DNS y **no mira /etc/hosts**.
3. `sudo ss -ntlp | grep :80` (o `sudo netstat -ntlp | grep :80`).
4. El 80 es un puerto privilegiado (<1024): solo root puede abrir un socket que escuche ahí.
5. ¿Tengo IP? (`ip addr`) → ¿llego al gateway/al host? (`ping`) → ¿resuelve el nombre? (`dig` / `/etc/hosts`) → ¿el puerto 80 está abierto? (`nc` / `nmap`) → ¿el servidor responde lo esperado? (`curl -v`).

Capítulo 2 — Red

1. A `192.168.1.33`: está en mi red (/24), le hablo directo, uso **su** MAC (ARP a .33). A `8.8.8.8`: está fuera, uso la MAC del **gateway** (.1); la IP de destino sigue siendo 8.8.8.8.
2. **filtered**: mandé el SYN y no llegó respuesta (ni SYN/ACK ni RST), o volvió un ICMP de error. Un firewall descartó el paquete.
3. Recibió **9** Time Exceeded (3 saltos × 3 sondas). Del destino recibió **Port Unreachable** (ICMP tipo 3).
4. En pampero no sos root, entonces no podés armar paquetes crudos: `-sT` usa la llamada `connect()` del sistema. En tu VM sos root y podés hacer SYN scan (`-sS`), más rápido.
5. `sudo ip route add 172.20.0.0/16 via 192.168.1.254`.

Capítulo 3 — HTTP

1. (a) 301, (b) 403, (c) 401, (d) 409 (o 412 si usaste un pedido condicional), (e) 400, (f) 415, (g) 503, (h) 405.
2. El **fragmento** (`#section-14.19`). No se envía: lo usa solo el navegador para posicionarse dentro de la página ya descargada; mandarlo no aportaría nada al servidor.
3. `YTpi` → `a:b` (usuario `a`, password `b`).
4. `curl -v -L -H 'Accept-Language: en' -H 'Accept: application/json' -i http://x.com/doc`.
5. 304 no trae cuerpo: el cliente reutiliza su copia en caché. Ahorra **ancho de banda** (no retransmite el recurso) y el servidor ahorra enviar el cuerpo. 200 retransmite todo.
6. Porque dice «usá tu copia hasta 1 hora *sin preguntar*; después revalidá». Durante esa hora sí usa la caché; «no-cache»/«no-store» serían lo contrario.

Capítulo 4 — nginx

1. `Host: c` → el **default_server** (no matchea `a` ni `b`). `curl http://a/` sin `a` en /etc/hosts → no resuelve el nombre (error de DNS); si resolviera a tu IP, iría al server `a`.
2. Con barra final en el proxy_pass (`.../`) y `location /api/`: `/api/users/7` → `http://127.0.0.1:3000/users/7` (se come `/api/`). Sin barra final: → `http://127.0.0.1:3000/api/users/7`.
3. Porque el backend `foo.leak.com.ar` usa virtual hosts y espera ver **su** nombre en el header Host para elegir el sitio. Con `$host` le llegaría `foo.example.org` y no lo reconocería.

4. `server { listen 80 default_server; server_name _; default_type text/plain; return 200 "What?\n"; }`
5. Redirect: `location / { return 301 http://foo.leak.com.ar$request_uri; }` (la URL del navegador cambia, por eso solo sirve cuando «puede cambiar»).
6. `curl --compressed -v ...` y mirás que la respuesta traiga `Content-Encoding: gzip` (y que bajen menos bytes que sin `--compressed`).

Capítulo 5 — TLS y certificados

1. Firma con su clave **privada**; el cliente verifica con la **pública** que está en el certificado.
2. Porque el asimétrico es lento: se usa una vez para acordar/proteger la clave de sesión, y después todos los datos van con cifrado **simétrico** (AES/ChaCha20), que es rápido.
3. ECDHE = intercambio de claves Diffie-Hellman efímero con curvas elípticas (forward secrecy); ECDSA = el certificado/firma del servidor es de curva elíptica; AES_256_GCM = cifrado simétrico de datos (256 bits, modo autenticado); SHA384 = hash.
4. Los navegadores modernos validan el nombre contra el **SAN** (Subject Alternative Name), no contra el CN. Sin SAN que incluya `foo`, lo rechazan aunque confíen en la CA.
5. Porque en una misma IP puede haber varios sitios HTTPS y el servidor necesita saber cuál antes de elegir qué certificado presentar; el header Host va cifrado, así que el nombre se adelanta en claro en el SNI del ClientHello.
6. `ssl_certificate` = parte pública (el certificado, que se muestra a todos). `ssl_certificate_key` = la clave **privada**, que **nunca** se comparte.
7. Porque el enunciado prohíbe opciones extra en curl (`-k` incluido). curl tiene que confiar de verdad en el certificado, así que hay que usar una CA que esté en el trust store (mkcert) o agregar el certificado al sistema.

Capítulo 6 — DNS

1. La respuesta **no es autoritativa**: vino de un resolver/caché (tiene `ra`, ofrece recursión), no del servidor dueño de la zona.
2. Al de **preferencia 5** (`b.mail.com`): menor número = mayor prioridad.
3. Porque un nombre con CNAME no puede tener otros registros, y el apex **siempre** tiene SOA y NS. CNAME en `@` chocaría con ellos.
4. stub→recursivo (recursiva). Recursivo→root→`ar.`→`uba.ar.`/`fi.uba.ar.` (iterativas, cada una devuelve un referral) hasta el autoritativo, que responde con `aa`. El recursivo cachea y te contesta.
5. `dig foo` pregunta al DNS configurado en tu sistema; `dig @127.0.0.1 foo` pregunta explícitamente a un servidor en tu máquina (tu BIND).
6. Los resolvers cachearon la IP vieja y la mantienen hasta que venza el TTL (86400 s = 1 día). Deberías haber **bajado el TTL** con anticipación antes del cambio.

Capítulo 7 — BIND9

1. Ejemplo:

```
$TTL 1h
@   IN SOA ns1.pdc.lab. root.pdc.lab. ( 2025103101 1h 15m 1w 5m )
@       IN NS    ns1.pdc.lab.
ns1     IN A     10.0.0.2
foo 10m IN A     10.0.0.5
bar     IN CNAME foo
@       IN MX 10 mx.pdc.lab.
mx      IN A     10.0.0.3
```

2. (a) `www.ejemplo.com.` (b) `www.ejemplo.com.ejemplo.com.` (le agrega el origen por no tener punto) (c) `www.ejemplo.com.` (ya es absoluto) (d) `ejemplo.com.`
3. Declaración: `zone "0.0.10.in-addr.arpa" { type master; file "/etc/bind/db.10.0.0"; };` y en ese archivo: `5 IN PTR foo.pdc.lab.` (con SOA y NS como cualquier zona).
4. `@ IN MX mail.pdc.lab.` → falta la **preferencia** (número). `ftp IN CNAME 10.0.0.9` → CNAME apunta a un **nombre**, no a una IP. `@ IN CNAME foo` → el apex **no** puede ser CNAME.
5. `named-checkzone <zona> <archivo>` (y `named-checkconf` para named.conf).

Capítulo 8 — Correo

1. Llega a `a@x.com` (manda el **RCPT TO**, el sobre). El usuario ve como remitente el `From:` de la carta, aunque el MAIL FROM sea otro.
2. `chau` = 99,104,97,117. 4 bytes → base64 ocupa 8 caracteres (`Y2hhdQ==` son 6 + 2 de padding = 8). `printf 'chau' | base64` → `Y2hhdQ==`.
3. `=C3=A9` son los bytes 0xC3 0xA9 = **é** en UTF-8.
4. Para que cada cliente muestre lo que puede: los que no renderizan HTML usan la versión `text/plain`; es la misma información en dos formatos (`multipart/alternative`).
5. Falla SPF (`~all` = softfail): no se rechaza de plano, pero puede marcarse como sospechoso o ir a spam, según la política DMARC del dominio.
6. Para no ser un open relay desde su red: si no bloquearan el 25 saliente, alguien podría usar la red del ITBA para mandar spam directo a MTAs externos.

Capítulo 9 — DHCP

1. Ejemplo:

```
subnet 10.1.1.0 netmask 255.255.255.0 {
    range 10.1.1.50 10.1.1.99;
    option routers 10.1.1.1;
    default-lease-time 120;
    max-lease-time 120;
}
```

2. Origen `0.0.0.0`, destino `255.255.255.255` (broadcast): el cliente todavía no tiene IP y no sabe la del servidor, así que no puede mandar unicast.
3. Renovación (T1) al 50% → **60 s**. Rebinding (T2) al 87,5% → **105 s**.
4. Para que esa IP nunca entre en la asignación dinámica: si la `fixed-address` estuviera dentro del `range`, el servidor podría dársela a otro cliente antes que a la impresora.
5. Con un bloque `host` para esa MAC y `deny booting;` (o `ignore booting;`): el servidor no le ofrece configuración.

Capítulo 10 — SSH

1. Abre el puerto 8080 en **tu** máquina; conectarte a `localhost:8080` te lleva, por el túnel, al puerto 80 de `web`. Ves el sitio de web como si fuera local.
2. Abre el 8080 en **server**; quien en server se conecte a `localhost:8080` llega, por el túnel, a tu `localhost:3000`. Sirve para exponer un servicio tuyo en el server.
3. `ssh -D 9090 user@pampero.itba.edu.ar` y luego `curl -x socks5h://127.0.0.1:9090 https://api.ipify.org/` (devuelve la IP de pampero).
4. Porque SSH se niega a usar una clave que otros puedan leer. Sin `chmod 600`: «*UNPROTECTED PRIVATE KEY FILE … permissions are too open*» y no se conecta.
5. Abre el 1234 local y lo tuneliza al 1234 de localhost, que es el mismo puerto recién abierto: apunta a sí mismo, no hay servicio real detrás.
6. Desde el ITBA el host interno se resuelve y el 1080 de pampero es alcanzable; desde casa tu DNS no resuelve ese nombre interno (y quizás el 1080 no esté expuesto). Con `socks5h` el nombre lo resuelve **pampero**, lo que soluciona la parte de DNS (si además el 1080 es accesible).

Capítulo 11 — Wireshark

1. `http.request && ip.dst == 10.0.0.5`.
2. Filtrar `smtp` y hacer clic derecho → **Follow → TCP Stream**.
3. En el campo **`yiaddr`** («Your client IP address»), en el mensaje **Ack** (también está en el Offer).
4. **traceroute**. Hay 3 saltos intermedios hasta el destino (los 3 Time Exceeded) más el destino que respondió Destination Unreachable → el destino está a 4 saltos.
5. Telnet va en texto plano; SSH cifra todo el canal, así que en el pcap solo se ven bytes cifrados.
6. `base64 -d imagen.txt > imagen.png` (o `cat imagen.txt | base64 -d > imagen.png`).

Chuleta final: comandos para tener a mano

Necesito...	Comando
Mi IP / MAC / gateway / ruta	`ip addr` · `ip route` · `route -n`
MAC de una IP vecina	`ping -c1 IP; arp -n | grep IP`
Puertos abiertos de un host	`nmap -sT -p- HOST` (sin root) · `sudo nmap -sS -sV -Pn HOST`
Routers del camino	`traceroute -n HOST`
Agregar ruta	`sudo ip route add RED/MASK via GW`
Resolver nombre / MX / TXT / SOA	`dig +short N` · `dig MX N` · `dig TXT N` · `dig @127.0.0.1 N`
HTTP con headers/redirect/proxy	`curl -v -L -H 'H: v' -x socks5h://host:1080 URL`
Probar virtual host sin DNS	`curl -H 'Host: foo' http://127.0.0.1/`
Validar configs	`nginx -t` · `named-checkzone Z F` · `dhcpd -t`
Reiniciar servicio y ver log	`sudo systemctl restart SVC; journalctl -u SVC -e`
Certificado autofirmado	`openssl req -x509 -newkey rsa:2048 -nodes -keyout k.key -out c.crt -subj "/CN=nombre" -addext "subjectAltName=DNS:nombre"`
SSH con llave / túnel local / dinámico	`ssh -i k.pem -p N u@h` · `ssh -L a:host:b u@h` · `ssh -D 9090 u@h`
Permisos de llave	`chmod 600 llave.pem`
base64	`... | base64` · `... | base64 -d`
pcap	`zcat x.pcap.gz > x.pcap`; en Wireshark: filtro + Follow TCP Stream

🗣 Lo último antes de entrar

Recordá la regla de oro de la corrección: **demostrar que sabés por qué funciona**. Para cada ejercicio reiniciá el servidor, mostrá que arranca sin errores, probalo con el cliente del protocolo (curl, dig, nc, nmap) mostrando cada requisito, y al final mostrá los archivos de config. Narrá qué hacés y por qué. Suerte. 🍀